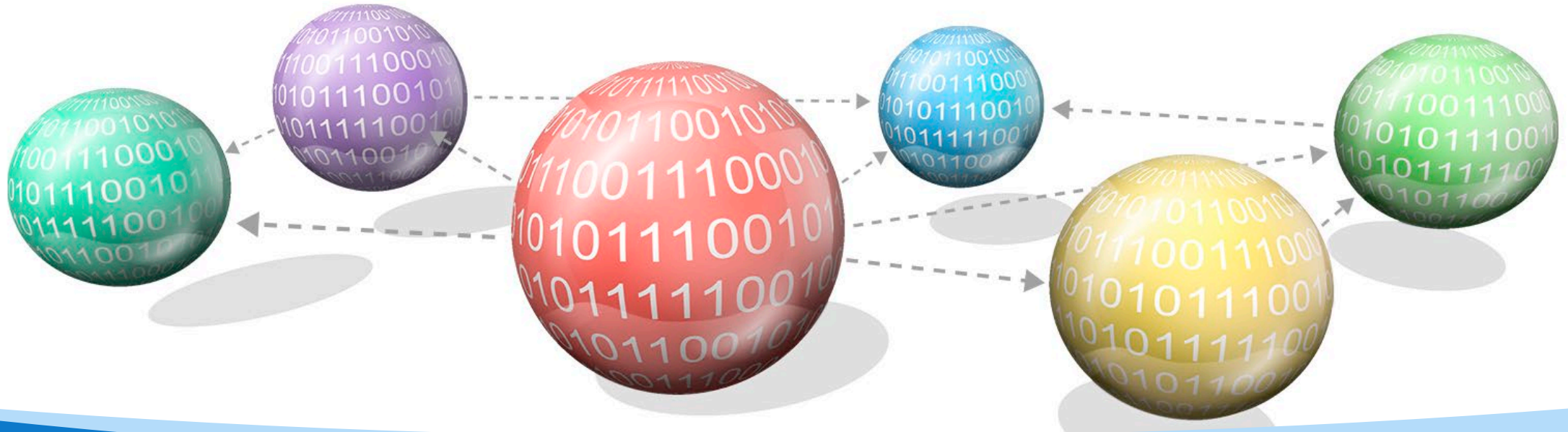




SC/CZ/CE2002 Object-Oriented Design & Programming

Chapter 10: OO Concepts in C++

Dr. Li Fang
Senior Lecturer, CCDS

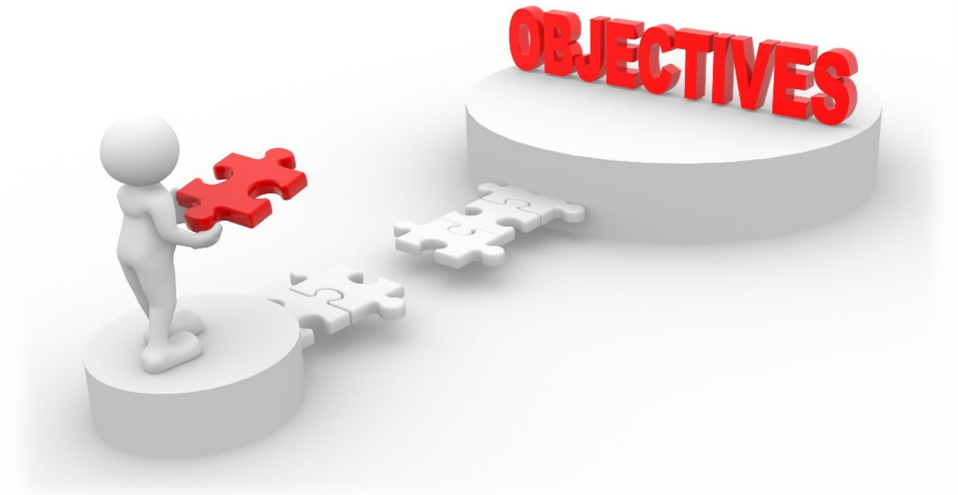


**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Objectives

By the end of this chapter, you should be able to:

- Explain the differences between C and C++
- Explain the differences between Java and C++



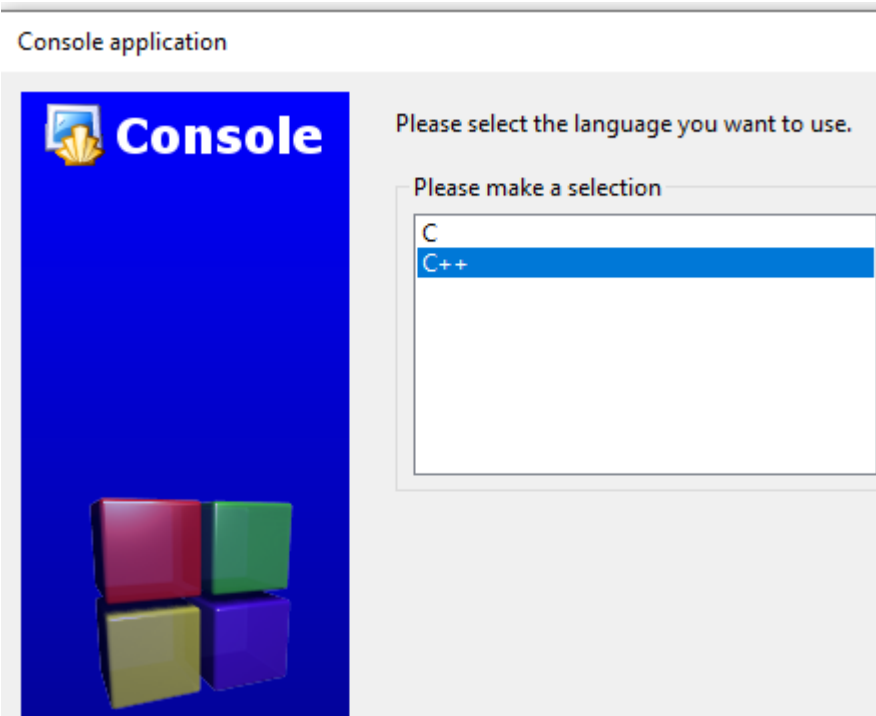
Why C++

- <https://www.educative.io/blog/learn-cpp-for-2022#cppuses>

Company
Microsoft
Google
Facebook
Amazon
Salesforce

- C++ is a great language to learn if you're a programmer who wants a **deep understanding of how computers work**.
- C++ gives you direct control over the system hardware and memory, which is why it's widely used for systems that require real-time responses or have tight performance constraints.
- There are always companies looking for developers with good C++ skills, given its use in high-performance software, product development, game development, and in embedded systems.

First C++ program (OOP is not enforced in C++)



```
#include <iostream>

//using namespace std;

int main()
{
    std::cout << "Hello world!" <<"2002" << std::endl;
    return 0;
}
```

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" <<"2002"<< endl;
    return 0;
}
```

The background of the slide features a dense, out-of-focus field of white, three-dimensional letters and numbers floating in the air. In the center of this field, the word "TOPIC" is prominently displayed in a large, bold, red, three-dimensional font. Below this graphic, a solid blue horizontal band spans the width of the slide.

TOPIC

Topic 1: namespace

C vs. C++: Simple Comparison

```
#include <stdio.h>
```

```
int getFact(int n) {  
    int c, fact = 1;  
    if ( n < 0 )  
        printf("Number should be non-negative.");  
    else  
        for (c = 1; c <= n; c++)  
            fact = fact * c;  
    return fact;  
}
```

```
int main()  
{  
    int n = 1;  
  
    printf("Enter a number to calculate it's factorial\n");  
    scanf("%d", &n);
```

```
    printf("Factorial of %d = %d\n", n, getFact(n));
```

```
    return 0;
```

```
}  
// save as <anyname>.c
```

```
#include <iostream>
```

```
using namespace std;
```

```
int getFact(int n) {  
    int c, fact = 1;  
    if ( n < 0 )  
        std::cout << "Number should be non-negative." << endl ;  
    else  
        for (c = 1; c <= n; c++)  
            fact = fact * c;  
    return fact;  
}
```

```
int main()  
{  
    int n = 1;
```

```
    cout << "Enter a number to calculate it's factorial > ";  
    cin >> n;
```

```
    cout << "Factorial of " << n << " = " << getFact(n) << endl;
```

```
    return 0;
```

```
}  
// save as <anyname>.cpp
```

std Namespace

- The identifiers of the C++ standard library are defined in a namespace called std.
- In order to use any identifier belonging to the standard library, we need to specify that it belongs to the std namespace.

Utilizing std Identifiers

We can utilize identifiers of the `std` namespace in our program with:

- the `::` operator
- the `using` declaration

<https://www.programiz.com/cpp-programming/std-namespace>

<https://www.codingunit.com/cplusplus-tutorial-namespaces-and-anonymous-namespaces>



What are the primary purposes of a namespace in C++?

① Start presenting to display the poll results on this slide.

Purpose of using Namespace

- In C++, namespaces are used to organize code into logical groups and to prevent name collisions that can occur especially when your code base includes multiple libraries.

```
namespace LibraryA {  
    void calculate() {  
        // Implementation  
    }  
}  
  
namespace LibraryB {  
    void calculate() {  
        // Different Implementation  
    }  
}
```

```
LibraryA::calculate(); // Calls the function from LibraryA  
LibraryB::calculate(); // Calls the function from LibraryB
```

Purpose of using Namespace cont'

- Namespaces help in logically grouping code together. This is especially useful in large projects where you might have hundreds of functions and classes. For example, if you are developing a game, you might have namespaces for rendering, physics, audio, etc.

```
namespace Audio {  
    void playSound() {  
        // Implementation  
    }  
}  
  
namespace Graphics {  
    void renderScene() {  
        // Implementation  
    }  
}
```

slido

Please download and install the
Slido app on all computers you use



How do you define a namespace in C++?

① Start presenting to display the poll results on this slide.



How do you bring all the members of a namespace into the current scope?

① Start presenting to display the poll results on this slide.

Namespace

```
// in example.h
#include <iostream>
using namespace std ;

namespace root {
    namespace sub {
        class Test {
        public :
            void print() {
                cout << "an example of namespace" << endl ;
            }
        };
    }
}
```

```
#include example.h
using namespace root::sub ;
int main() {
    Test t ;
    t.print();
}
```

```
#include example.h
using namespace root ;
int main() {
    sub::Test t ;
    t.print();
}
```

```
#include example.h
int main() {
    root::sub::Test t ;
    t.print();
}
```



Which of the following is a disadvantage of using using namespace std; in large projects?

① Start presenting to display the poll results on this slide.

Using the using Directive

You can use the using directive to avoid typing the namespace name every time you use an identifier from it. However, overuse of the using directive, especially in header files, can lead to name conflicts and is generally discouraged in large projects.

The background of the slide features a dense field of white, three-dimensional letters and numbers floating at various depths and angles, creating a sense of motion and complexity. Overlaid on this background is the word "TOPIC" in a large, bold, red 3D font. The letters of "TOPIC" are thick and have a slight shadow, making them stand out prominently from the chaotic letter background.

TOPIC

Topic 2: class structure

C++ class

keyword

user-defined name

class **ClassName**

{ Access specifier: //can be private,public or protected

Data members; // Variables to be used

Member Functions() { } //Methods to access data members

}; ← **// Class name ends with a semicolon**

<https://www.geeksforgeeks.org/c-classes-and-objects/>

Point
#x : int
#y : int
+Point(x : int, y : int)
+toString() : String
+setPoint(x : int, y : int)
+getX() : int
+getY() : int

```

public class Point {
    protected int x, y;

    public Point() { x = 0; y = 0; }
    public Point(int x, int y) {
        this.x = x; this.y = y;
    }

    public void setPoint(int x, int y) {
        this.x = x; this.y = y;
    }

    public String toString() {
        return "[" + x + "," + y + "]";
    }

    public int getX() {return x; }
    public int getY() {return y; }
}

```

slido



What is the default accessibility in Java?

① Start presenting to display the poll results on this slide.

slido



What is the default accessibility in C++?

① Start presenting to display the poll results on this slide.

access modifiers c++ Java comparison

Keyword	C#	C++	Java
<code>private</code>	class	class a  fr	class
<code>fileprivate</code>	-	-	-
<code>private protected</code>	derived classes in the same assembly	-	-
<code>protected internal</code>	same assembly and/or derived classes	-	-
<code>protected</code>	derived classes	derived classes and/or friend classes	derived classes and/or within same package
<code>package</code>	-	-	with  package
<code>internal</code>	same assembly	-	-
<code>public</code>	everybody	everybody	everybody
<code>open</code>	-	-	-

By default access to members of a C++ class is private.

The Default access modifier is package-private (i.e DEFAULT) and it is visible only from the same package.

Class Definition: Classes

```
class Point {  
    private:  
        int _x, _y;    // point coordinates, default private  
    public:  
        void setX(const int val); // declaration  
        void setY(const int val); // declaration  
        int getX() { return _x; } // declaration + implementation  
        int getY() { return _y; } // declaration + implementation  
};  
void Point::setX(const int val) { _x = val; }  
void Point::setY(const int val) { this->_y = val; }  
// scope operator "::"
```

- In C++, the separation of declaration and implementation is a fundamental design choice that offers several advantages.

Method defined inside or outside of class

```
#include <iostream>
using namespace std;

class Point{
    int x=0, y=0;
public:
    string toString(){
        string result = "["+to_string(x)+", "+to_string(y)+"]";
        return result;
    }
    int getX(){
        return x;
    } //declaration + implementation
    int getY(); //declaration only
};

int Point:: getY(){
    return y;
} //implementation outside
```



TOPIC

Topic 3: Object Creation in C++

Java: Heap-Only Object Allocation

- Java was designed with the goal of simplification and ease of use in mind

C++: Stack and Heap Object Allocation

- **C++ offers a more complex memory model that provides the programmer with greater control and efficiency.**

```
#include <iostream>

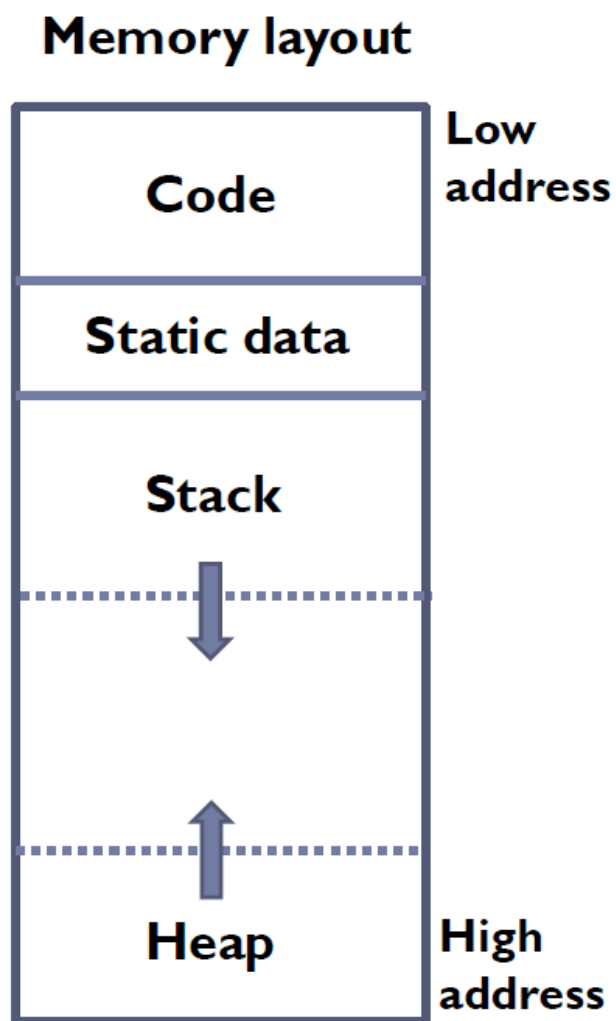
using namespace std;
class Point{
    int x=0, y=0;
public:
    string toString(){
        string result = "["+to_string(x)+", "+to_string(y)+" ";
        return result;
    }
    int getX(){
        return x;
    }
    int getY(){
        return y;
    }
};
```

```
int main()
{
    cout << "Hello world!" << endl;
    Point p;
    cout<<p.toString()<<p.getX()<<endl;
    Point* p2 = new Point();
    cout<<p2->toString()<<p2->getX()<<endl;
    delete p2;
    return 0;
}
```

```
Hello world!
[0,0]0
[0,0]0
```

Memory Layout of a Compiled Program

- ▶ Memory layout (for many languages)
 - ▶ Code area: fixed size and read only
 - ▶ Static data: statically allocated data
 - ▶ variables/constants
 - ▶ Stack: parameters and local variables of methods as they are invoked.
 - ▶ Each invocation of a method creates one **frame** which is pushed onto the stack
 - ▶ Heap: dynamically allocated data
 - ▶ class instances/data array
 - ▶ Stack and heap grow towards each other



Object Creation

Method 1 (stack)

```
Point aPoint; // Point::Point() Note : without ( )  
Point bPoint(12, 34); // Point::Point(const int, const int)  
    aPoint.setY(20);  
    int y = aPoint.getY();
```



Stack	
bPoint	12
	34
aPoint	0
	20

Lifetime: The object exists until the block in which it is defined goes out of scope.

Memory Management: Automatic. You don't need to explicitly deallocate memory.

Safety: Stack objects are safe from memory leaks as they are automatically destroyed.

Speed: Stack allocation is generally faster than heap allocation.

Size Limitations: Stack space is limited (usually a few MBs), so large objects should not be allocated on the stack.

Object Creation

Method 2 (heap)

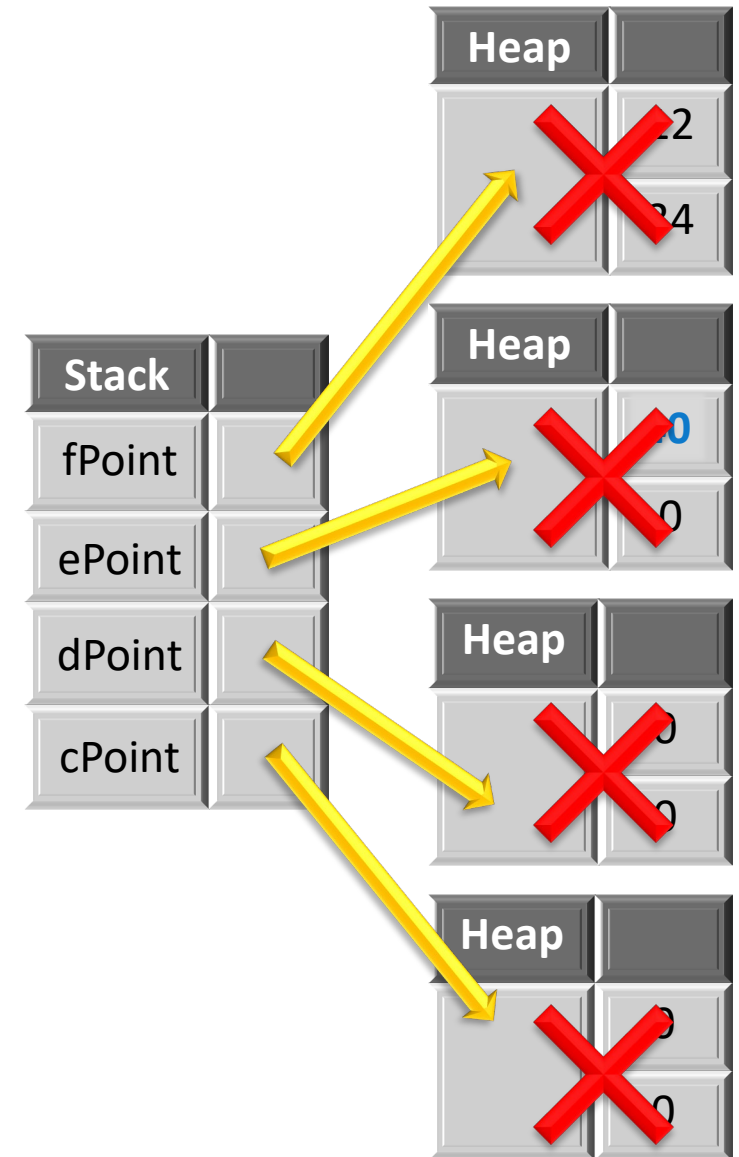
```
Point *cPoint = new Point(); // Point::Point()
Point *dPoint = new Point(); // Point::Point()
Point *ePoint;
    ePoint = new Point();      // Point::Point()
Point *fPoint = new Point(12, 34); // Point::Point(const int,
                                   // const int)
ePoint->setX(40);
int x = ePoint->getX();
delete cPoint, dPoint, ePoint, fPoint;
```

Lifetime: The object exists until it is explicitly destroyed using delete.
Memory Management: Manual. The programmer must explicitly deallocate memory to avoid memory leaks.

Safety: Objects allocated on the heap are prone to memory leaks if not managed carefully.

Speed: Heap allocation involves internal management which makes it slower compared to stack allocation.

Size Limitations: The heap is much larger than the stack, allowing for the allocation of large objects.





How is an object of a class created in C++?

- ① Start presenting to display the poll results on this slide.

Class Destructors

A destructor is a member function that is invoked automatically when the object goes out of scope or is explicitly destroyed by a call to delete. A destructor has the same name as the class, preceded by a tilde (~). For example, the destructor for class String is declared: ~String().

If you do not define a destructor, the compiler will provide a default one; for many classes this is sufficient. You only need to define a custom destructor when the class stores handles to system resources that need to be released, or pointers that own the memory they point to.

```
#include <iostream>
using namespace std;
class A
{
public:
    A() {x=0; cout<<"A default"<<x; }
    A(int x_) { x=x_; cout<<"A parameter"<<x; }
    ~A() { /* do clean up */ } // Destructor

    int x;
};

class Point{
protected:
    A a;
public:
    Point() {a=A(7); }
    ~Point() { /* do clean up */ } // Destructor
};
```



What is the significance of garbage collection in Java compared to C++?

① Start presenting to display the poll results on this slide.



TOPIC

Topic 4: Constructor and Initializer list

Constructor

Hello world!

[1,1]1

[6,9]6

[1,1]1

[7,8]7

```
#include <iostream>
using namespace std;

class Point{
protected:
    int _x, _y;
public:
    Point() {_x =1; _y =1;};
    Point(int x, int y){
        _x = x; _y = y;
    };
    string toString(){
        string result = "["+to_string(_x)+"," +to_s
        return result;
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
int main()
{
    cout << "Hello world!" << endl;
    Point p1;
    cout<<p1.toString()<<p1.getX()<<endl;
    Point p2(6,9);
    cout<<p2.toString()<<p2.getX()<<endl;
    Point* p3 = new Point();
    cout<<p3->toString()<<p3->getX()<<endl;
    Point* p4 = new Point(7,8);
    cout<<p4->toString()<<p4->getX()<<endl;
    delete p3;
    delete p4;
    return 0;
}
```



What is an initializer list in C++ primarily used for?

① Start presenting to display the poll results on this slide.

Initialization of base classes and members (C++ only)

Constructors can initialize their members in two different ways.

1. Assignment way (assignment statement in constructor's method body):

A constructor can use the arguments passed to it to initialize member variables in the constructor definition:

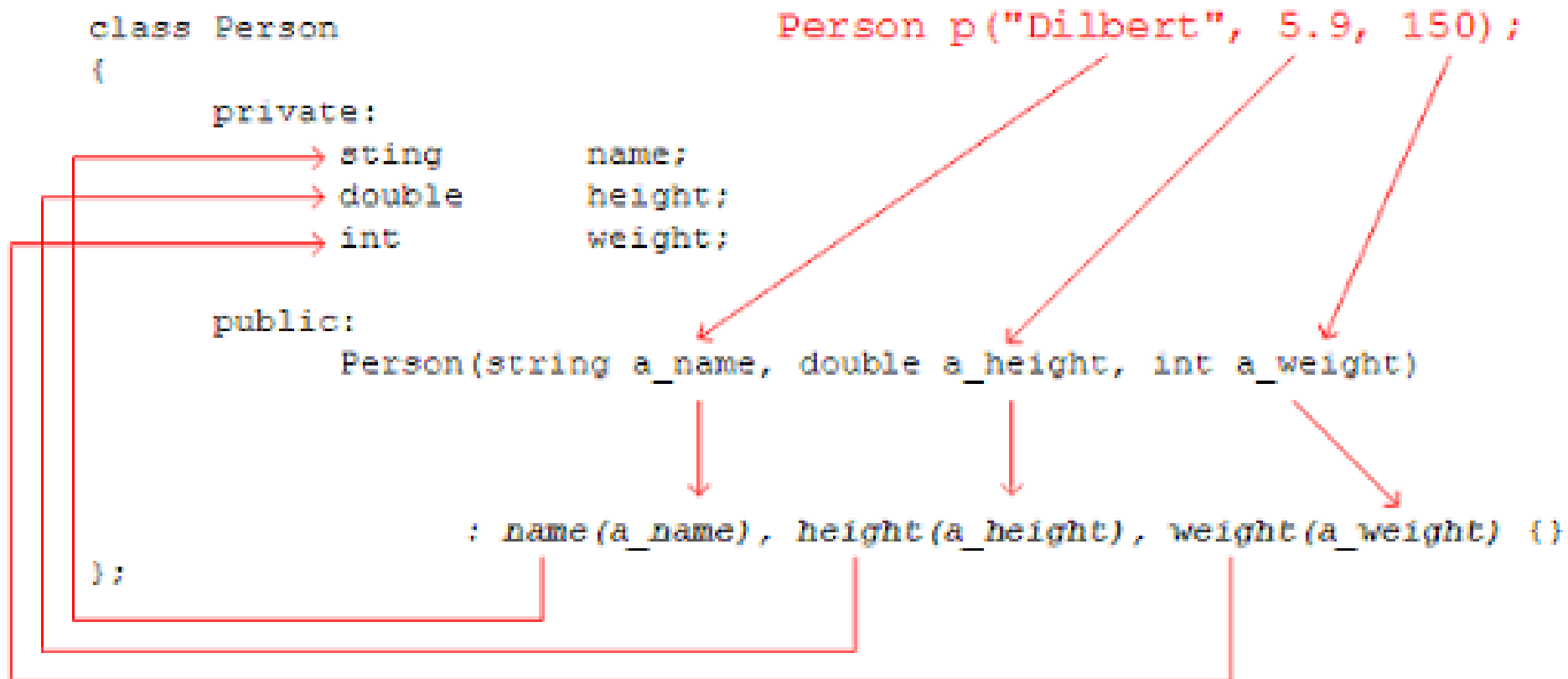
```
complex(double r, double i = 0.0) { re = r; im = i; }
```

2. **Initializer List way** : a constructor can have an initializer list within the definition but prior to the constructor body:

```
complex(double r, double i = 0) : re(r), im(i) { /* ... */ }
```

Initializer list

The initializer list is used to directly initialize data members of a class. An initializer list starts after the constructor name and its parameters. The list begins with a colon (:) and is followed by the list of variables that are to be initialized – all of the variables are separated by a comma with their values in brackets.



Initializer list

```
class Point{
protected:
    int _x, _y;
public:
    Point() {_x =1; _y =1;};
    Point(int x, int y):_x(x), _y(y){
    };
    string toString(){
        string result = "["+to_string(_x)+"," +to_string(_y)+"]";
        return result;
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

What is output of the following code?

```
#include <iostream>
using namespace std;
class A
{
public:
    A() {x=0; cout<<"A default"<<x; }
    A(int x_) { x=x_; cout<<"A parameter"<<x; }
    int x;
};
class Point{
protected:
    A a;
public:
    Point() {a=A(7); }
};
int main()
{
    Point p;
}
```

slido

Please download and install the
Slido app on all computers you use



What is the output of previous code?

① Start presenting to display the poll results on this slide.

What is output of the following code?

```
#include <iostream>
using namespace std;
class A
{
public:
    A() {x=0; cout<<"A default"<<x; }
    A(int x_) { x=x_; cout<<"A parameter"<<x; }
    int x;
};
class Point{
protected:
    A a;
public:
    Point() {a=A(7); }
};
int main()
{
    Point p;
}
```

A. A default0

B. A parameter7

 C. A default0A parameter7



What is the advantage of using an initializer list over assignment inside the constructor body?

① Start presenting to display the poll results on this slide.

Using initializer lists in C++ is a very efficient way to initialize class members. When you use an initializer list, the members are initialized directly with the given values, rather than being default-initialized first and then assigned a value.

- For non-primitive data types, using an initializer list avoids calling the default constructor

What is output of the following code?

```
#include <iostream>
using namespace std;
class A
{
public:
    A() {x=0; cout<<"A default"<<x; }
    A(int x_) { x=x_; cout<<"A parameter"<<x; }
    int x;
};

class Point{
protected:
    A a;
public:
    Point():a(A(7)){}
};

int main()
{
    Point p;
}
```

- A. A default0
- ✓ B. A parameter7
- C. A default0A parameter7



TOPIC

Topic 5: Class Inheritance

```
public class Circle extends Point {
    private double radius;
    public Circle() { radius = 1; }
    public Circle(double radius) { this.radius = radius; }
    public Circle(double radius, int a, int b) {
        super(a,b); this.radius = radius;
    }
    public double getRadius() { return radius; }
    public void setRadius(double radius) { this.radius = radius; }
    public double area() {
        return Math.PI * Math.pow(radius,2);
    }
    public String toString() {
        return "Circle of radius "+
            radius+" at point ["+x+", "+y+"]";
    }
}
```

slido

Please download and install the
Slido app on all computers you use



What members of a base class can be inherited by a derived class in C++, assuming inheritance works similarly to Java?

① Start presenting to display the poll results on this slide.

Class Inheritance

General Form :



```
class derived-class-name : [visibility-mode] base-class-name
{
    .....
};
```

visibility-mode : [optional] **private** / **public**

private : privately inherited (default)

 ‘**public** members’ of **base** class become

 ‘**private** members’ of derived class.

public : publicly inherited

 ‘**public** members’ of **base** class remain

 ‘**public** members’ of derived class.

```
Base
public : int data()
```

```
class Sub : Base
```

```
Sub
private : int data()
```

slido

Please download and install the
Slido app on all computers you use



If a class B is inherited from class A using protected visibility mode, what happens to the public members of A in B?

① Start presenting to display the poll results on this slide.

The inheritance in Java is always *public inheritance* by default

public, protected and private inheritance in C++

public, protected, and private inheritance have the following features:

public inheritance makes public members of the base class public in the derived class, and the protected members of the base class remain protected in the derived class.

protected inheritance makes the public and protected members of the base class protected in the derived class.

private inheritance makes the public and protected members of the base class private in the derived class.

Note: private members of the base class are inaccessible to the derived class.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y) {
    }
    int getX() {
        return _x;
    }
    int getY() {
        return _y;
    }
};
```

```
class Circle:Point{
    int _r;
public:
    Circle() {_r =1;}
};

int main()
{
    Circle c1;
    cout<<c1.getX()<<endl;
}
```

slido



What is the output of the previous code?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:Point{
    int _r;
public:
    Circle() {_r =1;}
};

int main()
{
    Circle c1;
    cout<<c1.getX()<<endl;
}
```

A 0

error: 'int Point::getX()' is inaccessible within this context



C. error

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
};

int main()
{
    Circle c1;
    cout<<c1.getX()<<endl;
}
```

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y) {
    }
    int getX() {
        return _x;
    }
    int getY() {
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
};

int main()
{
    Circle cl;
    cout<<cl.getX()<<endl;
}
```

- A. 0
-  B. 1
- C. error

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){
    }
    string toString(){
        string result = "["+to_string(_x)+", "+to_string(_y)+"]";
        return result;
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

Class Inheritance and initializer list

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        string result = "["+to_string(getX())+", "+to_string(getY())+", "+to_string(getR())+"]";
        return result;
    }
    int getR(){
        return _r;
    }
};
```

```
int main()
{
    cout << "Hello world!" << endl;
    Circle c1;
    cout<<c1.toString()<<c1.getX()<<endl;
    Circle *c2 = new Circle(7,8,9);
    cout<<c2->toString()<<c2->getX()<<endl;
    delete c2;
    return 0;
}
```

Class Inheritance

```
class Point3D : public Point {  
    int _z;  
public:  
    Point3D( const int x, const int y, const int z) : Point(x, y)  
        { _z = z; }  
.....  
};
```

Using Base class constructor



```
: Point(x, y) , _z(z) { }
```

Specify the desired base constructors after a **single colon** just before the body of constructor.

Using an initializer list avoids calling the default super class constructor

slido

Please download and install the
Slido app on all computers you use



What happens if an initializer list is not used in a derived class constructor, and the base class has no default constructor?

① Start presenting to display the poll results on this slide.

Initializer list

```
#include <iostream>
#include <string>

// Superclass without a default constructor
class SuperClass {
public:
    // Constructor that requires an argument
    SuperClass(const std::string& value) : value_(value) {
        std::cout << "SuperClass constructed with value: " << value << std::endl;
    }

private:
    std::string value_;
};

// Subclass must call one of the SuperClass's constructors
class SubClass : public SuperClass {
public:
    // SubClass's constructor must call SuperClass's constructor in its initializer list
    SubClass(const std::string& value) : SuperClass(value) {
        std::cout << "SubClass constructed with value: " << value << std::endl;
    }
};
```

SubClass in this example, cannot rely on a default constructor being automatically invoked for SuperClass. Instead, SubClass must explicitly call the constructor of SuperClass in its own constructor's initializer list.

Class Multiple Inheritance

Class Multiple Inheritance

```
class DrawableString : public Point, public DrawableObject  
{  
.....  
}
```

Class Multiple Inheritance example

```
#include <iostream>

// Base class A
class A {
public:
    A() { std::cout << "Constructor of A called" <<
std::endl; }
    void doSomethingA() { std::cout << "A doing
something" << std::endl; }
};

// Base class B
class B {
public:
    B() { std::cout << "Constructor of B called" <<
std::endl; }
    void doSomethingB() { std::cout << "B doing
something" << std::endl; }
};
```

```
// Subclass C inheriting from both A and B
class C : public A, public B {
public:
    C() { std::cout << "Constructor of C called" <<
std::endl; }
    void doSomethingC() {
        doSomethingA(); // Calling a function from A
        doSomethingB(); // Calling a function from B
        std::cout << "C doing something" << std::endl;
    }
};
```

```
int main() {
    C myObject;
    myObject.doSomethingC(); // Calls a
function from C, which in turn calls functions
from A and B
    return 0;
}
```



How can you resolve method ambiguity in C++ multiple inheritance?

① Start presenting to display the poll results on this slide.

```
#include <iostream>
using namespace std;

class Base1 {
public:
    void display() {
        cout << "Display from Base1" << endl;
    }
};

class Base2 {
public:
    void display() {
        cout << "Display from Base2" << endl;
    }
};
```

```
class Derived : public Base1, public Base2 {
public:
    void show() {
        // Resolving ambiguity by specifying the base
        class
            Base1::display(); // Calls Base1's display
        method
            Base2::display(); // Calls Base2's display
        method
    }
};

int main() {
    Derived obj;
    obj.show(); // Calls both display methods
    return 0;
}
```

Summary: Visibility Modes

- **Java:** Inheritance visibility is always **public**.
- Access to inherited members is controlled by the member's access modifiers (public, protected, default, and private), but there is no explicit way to define the inheritance visibility of a class.
- **C++:** You can specify the inheritance visibility using **public**, **protected**, or **private** when inheriting a class. This determines how the base class members are inherited in the derived class:
 - public inheritance keeps public and protected members the same.
 - protected inheritance makes public members protected.
 - private inheritance makes public and protected members private.

Summary: Initializer Lists

- Java:** Java does **not support initializer lists** in constructors. Instead, constructors use traditional assignment within the constructor body to initialize fields. If a superclass needs to be initialized, the subclass constructor must call the superclass constructor explicitly using the `super()` keyword before performing additional initialization.
- C++:** C++ **supports constructor initializer lists**, which allow direct initialization of member variables and base class constructors before the constructor body is executed. This is particularly useful for:
 - Initializing constants** and **reference** members, which must be initialized at the point of creation.
 - Calling the constructor of the superclass** (or any base class) directly in the initializer list. This provides a clean and efficient way to initialize the base class members before performing further initialization in the derived class.

Summary: Multiple Inheritance

- Java**: Supports **multiple inheritance** only through **interfaces**. A class can implement multiple interfaces, but it cannot inherit from multiple classes (only single class inheritance).
- C++**: Supports **multiple inheritance** of both **classes** and **interfaces** (abstract classes). A derived class can inherit from more than one base class.



TOPIC

Topic 6: Polymorphism



TOPIC

Topic 6.1: Method Overloading

Class **Method** Overloading

- Just like in Java!

```
int Add(int nX, int nY); // integer version
```

```
double Add(double dX, double dY); // floating point version
```

```
int Add(int nX, int nY, int nZ)
```

```
int GetRandomValue();
```

```
double GetRandomValue();
```

NOT Overloading

Class **Function** Overloading

- A **default parameter** is a function parameter that has a default value provided to it. Example,

```
void PrintValues(int nValue1, int nValue2=10)
{
    cout << "1st value: " << nValue1 << endl;
    cout << "2nd value: " << nValue2 << endl;
}

int main()
{
    PrintValues(1); // nValue2 will use default parameter of 10
    PrintValues(3, 4); // override default value for nValue2
}
```

slido



What is the output of the following code?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

void printValues(int v1=1, int v2)
{
    cout<<v1<<v2<<endl;
}

int main()
{
    printValues(3,4);
}
```

.....
error: default argument missing for parameter 2 of 'void printValues(int, int)'
.....

A. 34

B. 13

C. 14

 D. error

Class **Function** Overloading

- **Rules:**

- 1) All default parameters must be the **rightmost** parameters. The following is not allowed:

```
void PrintValue(int nValue1=10, int nValue2); // not allowed
```

```
void PrintValues(int nValue1, int nValue2=10) //correct
```

- 2) The **leftmost** default parameter should be the one most likely to be changed by the user.

```
#include <iostream>
using namespace std;

void printValues(int v1, int v2=1)
{
    cout<<v1<<v2<<endl;
}

void printValues(int v1)
{
    cout<<v1<<endl;
}
int main()
{
    printValues(3);
}
```

slido



What is the output of the previous code?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

void printValues(int v1, int v2=1)
{
    cout<<v1<<v2<<endl;
}

void printValues(int v1)
{
    cout<<v1<<endl;
}

int main()
{
    printValues(3);
}
```

error: call of overloaded 'printValues(int)' is ambiguous

A. 31
B. 13
C. 1
✓ D. error

Class **Function** Overloading

- **Constraints:**

- Do NOT count default parameters towards the parameters that make the function unique. Example,

`void PrintValues(int nValue);` **PrintValues(3) ???**
~~————~~ **NOT ALLOWED !!**
`void PrintValues(int nValue1, int nValue2=20);`

NOT Overloading



TOPIC

Topic 6.2: Method Overriding

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};

class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Circle c1;
    cout<<c1.toString()<<c1.getX()<<endl;
    return 0;
}
```

slido



What is the output of previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Circle cl;
    cout<<cl.toString()<<cl.getX()<<endl;
    return 0;
}
```

- A. Point1
-  B. Circle1
- C. Point0
- D. Circle0
- E. error

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Point *p2 = new Circle(7,8,9);
    cout<<p2->toString()<<endl;
    delete p2;
    return 0;
}
```

slido



What is the output of previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Point *p2 = new Circle(7,8,9);
    cout<<p2->toString()<<endl;
    delete p2;
    return 0;
}
```

- ✓ A. Point
- B. Circle
- C. error

slido

Please download and install the
Slido app on all computers you use



In C++, if a function in a base class is not declared as virtual, what happens when a derived class overrides it?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Point *p2 = new Circle(7,8,9);
    cout<<p2->toString()<<endl;
    delete p2;
    return 0;
}
```

What is the output of the following code?

```
#include <iostream>
using namespace std;
```

```
class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};
```

Without virtual => non polymorphic (no dynamic binding)

```
int main()
{
    Point *p2 = new Circle(7,8,9);
    cout<<p2->toString()<<endl;
    delete p2;
    return 0;
}
```

A. Point

✓ B. Circle

C. error

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};

int main()
{
    Point *p2 = new Cylinder();
    cout<<p2->toString()<<endl;
    delete p2;
    return 0;
}
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

class Cylinder:public Circle{
    int _r;
public:

    string toString(){
        return "Cylinder";
    }
    int getR(){
        return _r;
    }
};
```

slido



What is the output of previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;
```

```
class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};
```

If a method is declared virtual in a class, it is automatically virtual in all derived classes

```
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Cylinder:public Circle{
    int _r;
public:
    string toString(){
        return "Cylinder";
    }
    int getR(){
        return _r;
    }
};
```

```
{
    Point *p2 = new Cylinder();
    cout<<p2->toString()<<endl;
    delete p2;
    return 0;
}
```

A. Point

B. Circle

✓ C. Cylinder

D. error

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Circle c1;
    Point p1 = c1;
    cout<<p1.toString()<<endl;
    return 0;
}
```

slido



What is the output of previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};
```

```
int main()
{
    Circle c1;
    Point p1 = c1;
    cout<<p1.toString()<<endl;
    return 0;
}
```

Virtual function magic only operates on pointers(*) and references(&).

- ✓ A. Point
- B. Circle
- C. error



Which of the following allows
function overriding to work
correctly at runtime?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Circle c1;
    Point *p1 =&c1;
    Point *p2 = new Circle(1,2,3);
    cout<<p1->toString()<<p2->toString()<<endl;
    return 0;
}
```

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Circle c1;
    Point *p1 =&c1;
    Point *p2 = new Circle(1,2,3);
    cout<<p1->toString()<<p2->toString()<<endl;
    return 0;
}
```

A. PointCircle

B. CirclePoint

C. PointPoint

 D. CircleCircle

E. error

slido

Please download and install the
Slido app on all computers you use



If you create an object of a derived class on the stack and call a virtual function through a base class pointer, what will happen?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString()=0;
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

int main()
{
    Circle c1;
    Point p1 = c1;
    cout<<p1.toString()<<endl;
    return 0;
}
```

slido



What is the output of previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;
```

```
class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString()=0;
    int getX(){
        return _x;
    }
};
```

error: cannot allocate an object of abstract type 'Point'

note: because the following virtual functions are pure within 'Point':

```
};
note: 'virtual std::__cxx11::string Point::toString()'

```

error: cannot declare variable 'p1' to be of abstract type 'Point'

```
Point p1 = ci;
cout<<p1.toString()<<endl;
return 0;
```

```
}
```

A. Point

B. Circle

 C. error

Polymorphism

■ Virtual

- To force method evaluation to be based on object type rather than reference type. [<ref type> <name> = new <obj type>(..)]
- Without **virtual** => non polymorphic (no dynamic binding)
- **Example** : **virtual** void area() { cout << "....." << endl ; }
- **Virtual** function magic only operates on **pointers(*)** and **references(&)**.
- If a method is declared **virtual** in a class, it is **automatically virtual** in all **derived** classes.

■ Pure method => **abstract** method (pure virtual)

- By placing "**= 0**" in its declaration
- **Example** : **virtual** void area() = 0 ; // abstract method
- The class becomes an **abstract class**

Java vs C++

Feature	Java	C++
Default Overriding	All instance methods are virtual by default.	Methods are non-virtual by default; must be explicitly marked <code>virtual</code> for overriding.
Need for <code>virtual</code> keyword	Not required.	Required to mark a method as virtual for dynamic dispatch.
Polymorphism Requirements	Just using base class reference or object is enough.	Must use base class pointers or references to enable polymorphism.
Performance Consideration	All methods incur virtual function overhead.	Only virtual methods have dynamic dispatch overhead, giving more control over performance.
Method Dispatch	Dynamic (virtual) dispatch by default.	Static dispatch unless marked <code>virtual</code> .
Multiple Inheritance	No support for multiple inheritance (for classes).	Supports multiple inheritance, which complicates method dispatch.

The background of the top half of the slide features a dense, out-of-focus arrangement of 3D block letters in various shades of gray. The letters are scattered across the frame, creating a sense of depth and movement. In the center of this letter field, the word "TOPIC" is prominently displayed in a bright red, 3D, sans-serif font. The letters of "TOPIC" are slightly larger and more defined than the background letters, making them the focal point of the graphic.

TOPIC

Topic 6.3: dynamic_cast

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
int main()
{
    Point *p2 = new Circle();
    Cylinder* c2 = (Cylinder*) p2;
    cout<<c2->toString()<<c2->getH()<<endl;
    delete p2;
    return 0;
}
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

class Cylinder:public Circle{
    int _h = 5;
public:
    Cylinder() {_h = 5;}
    string toString(){
        return "Cylinder";
    }
    int getH(){
        cout<<"getH";
        return _h;
    }
};
```

slido



What is the output of previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y){_x =x; _y =y;}
    virtual string toString() const{
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
```

Downcasting to a derived class of real object type has no run time error, the method will call the real object type's method, those method defined in subclass of real object type will get unexpected result

```
Circle();
Cylinder* p2;
p2->getH();
```

- A. Cylinder
- B. CirclegetH5
- ✓ C. CirclegetH-1 (unexpected behavior during runtime)
- D. Runtime error

Downcasting: Java vs C++

Java ensures safer downcasting with built-in runtime checks, while C++ provides more control with both `dynamic_cast` (safe, but requires a polymorphic base) and `static_cast` (potentially unsafe, no runtime checks).

- Unsafe downcasting without built-in runtime type checking (unless you use `dynamic_cast`).
- C++ allows downcasting using `static_cast`, but it doesn't perform runtime type checks by default, so no protection is provided against invalid casts.
- If you want runtime type checking, you need to use `dynamic_cast`, which checks if the base class pointer can be safely cast to the derived class type. If the cast is invalid, `dynamic_cast` returns `nullptr` for pointers or throws an exception for references.

dynamic_cast

- Safe **down-cast**
 - Use **dynamic_cast**
 - `Type* t = dynamic_cast<Type*> ( variable )`
 - Returns NULL if the conversion was not possible
 - Only **applicable to pointers**
- If used very often, there is a good chance of your design being flawed

safely downcast; i.e., cast a base class pointer (or reference) to a derived class pointer (or reference). It can also be used for upcasting; i.e., casting a derived class pointer (or reference) to a base class pointer (or reference).

The type which the pointer needs be casted into.

```
new_type *newPtr = dynamic_cast<new_type*>(Ptr);
```

The pointer to be casted.

To use `dynamic_cast<new_type*>(ptr)` the base class should contain at least one virtual function.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
int main()
{
    Point *p2 = new Circle();
    Cylinder* c2 = dynamic_cast<Cylinder*> (p2);
    if (c2!=NULL){
        cout<<c2->toString()<<c2->getH()<<endl;
        cout<<"downcast"<<endl;
    }
    else cout<<"downcast invalid"<<endl;

    delete p2;
    return 0;
}
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

class Cylinder:public Circle{
    int _h = 5;
public:
    Cylinder() {_h = 5;}
    string toString(){
        return "Cylinder";
    }
    int getH(){
        cout<<"getH";
        return _h;
    }
};
```

slido



What is the output of previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y) {}
    virtual string toString() {
        return "Point";
    }
    int getX() {
        return _x;
    }
    int getY() {
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r) {}
    Circle(int r, int x, int y):Point(x,y),_r(r) {}
    string toString() {
        return "Circle";
    }
    int getR() {
        return _r;
    }
};

class Cylinder:public Circle{
    int _h = 5;
public:
    Cylinder() {_h = 5;}
    string toString() {
        return "Cylinder";
    }
    int getH() {
        cout<<"getH";
        return _h;
    }
};
```

```
int main()
{
    Point *p2 = new Circle();
    Cylinder* c2 = dynamic_cast<Cylinder*> (p2);
    if (c2!=NULL) {
        cout<<c2->toString()<<c2->getH()<<endl;
        cout<<"downcast"<<endl;
    }
    else cout<<"downcast invalid"<<endl;

    delete p2;
    return 0;
}
```

- A. CylindergetH5
- B. CirclegetH5
- C. CirclegetH-1 (unexpected behavior during runtime)
- ✓ D. downcast invalid
- E. Runtime error



TOPIC

Topic 7: Reference (&) vs. Pointer (*)

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};
```

```
int main()
{
    Circle p2(5,7,9);
    Point *pPtr = &p2 ;
    Point &pRef = p2 ;
    Point pVal = p2 ;

    cout<<pPtr->toString();
    cout<<pRef.toString();
    cout<<pVal.toString();

    return 0;
}
```

slido



What is the output of previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString(){
        return "Point";
    }
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};
```

```
int main()
{
    Circle p2(5,7,9);
    Point *pPtr = &p2 ;
    Point &pRef = p2 ;
    Point pVal = p2 ;

    cout<<pPtr->toString();
    cout<<pRef.toString();
    cout<<pVal.toString();

    return 0;
}
```

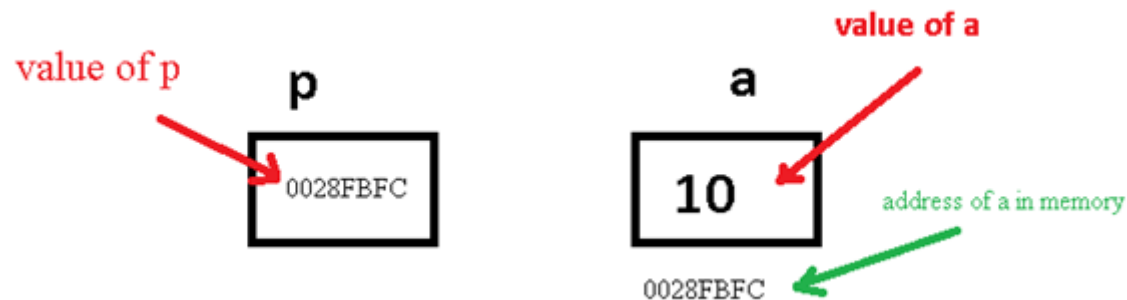
- ✓ A. CircleCirclePoint
- B. CircleCircleCircle
- C. CirclePointPoint

Virtual function magic only operates on pointers(*) and references(&).

pointer in C

```
int a;  
cout << "The address of \"a\" variable is " << &a << endl;  
  
//pointer to int  
int* p;  
  
p = &a; // now p is equal to the address of 'a' variable  
cout << "The value of \"p\" variable is " << p;
```

int *p p is a pointer points to an integer





Which of the following best
defines a pointer in C++?

① Start presenting to display the poll results on this slide.

pointer in C cont'

If you want to get the value stored in the memory by using a pointer, you have to **use unary dereference operator (*) in front of the pointer identifier**. This operator will return the value stored in the specified address.

```
int a = 10;
//pointer to int
int* p;
p = &a; // now p is equal to the address of 'a' variable
cout << "The value stored in memory location " << p << " is " << *p << endl;
```

A pointer object of a class uses “→” to access its data members and member functions

```
Circle c1;
Point *p1 = &c1;
Point *p2 = new Circle(1,2,3);
cout << p1->toString() << p2->toString() << endl;
```

From C to C++

■ References-syntactic sugar, code is easier to read and write

- “aliases” to “real” variables or objects
- Ampersand (&) is used to define a reference
- Cannot be NULL eg, `int &r;`
- Example,

```
int ix; /* ix is "real" variable */
```

```
int &rx = ix; /* rx is "alias" for ix */
```

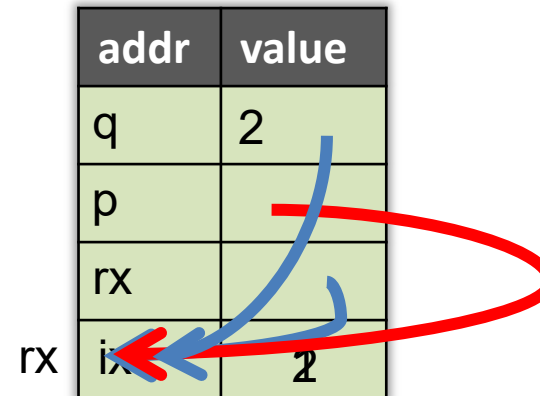
```
ix = 1; /* also rx == 1 */
```

```
rx = 2; /* also ix == 2 */
```

```
int *p = &ix; // addressOf ix assigned to pointer p.
```

```
int &q = *p; // dereference pointer p and assign to q as alias.
```

```
int q = *p;
```





Which of the following best defines a reference in C++?

① Start presenting to display the poll results on this slide.

slido



What is the output of the following code?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;

    int& ref = x;

    ref = 20;
    cout << "x = " << x ;

    x = 30;
    cout << " ref = " << ref << '\n';

    return 0;
}
```

- ✓ A. x = 20 ref = 30
- B. x = 10 ref = 20
- C. x = 20 ref = 20
- D. error

While calling a function, there are two ways that

Sr.No	Call Type & Description
1	Call by Value  This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.
2	Call by Pointer  This method copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.
3	Call by Reference  This method copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

By default, C++ uses call by value to pass arguments.

Reference (&) vs. Pointer (*)

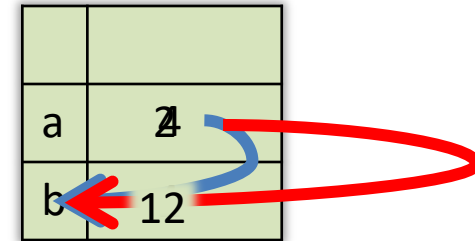
```
#include <iostream>
using namespace std ;

void addIt(int a) { a += a ; }
void doubleIt(int &a) { a *= 2 ; }
void tripleIt(int *a) { *a *= 3 ; }

int main() {
    int b = 2 ;
    cout << "b is now " << b << endl;
    addIt(b) ;
    cout << "after addIt(int a), b is " << b << endl;
    doubleIt(b) ;
    cout << "after doubleIt(int &a), b is " << b << endl;
    tripleIt(&b) ;
    cout << "after tripleIt(int *a), b is " << b << endl;
    // cin >> b;
}
```

OUTPUT:

b is now 2
after addIt(int a), b is 2
after doubleIt(int &a), b is 4
after tripleIt(int *a), b is 12



slido



What is the output of the following code?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

void fn (int& first, int& second)
{
    int temp = first;
    first = second;
    second = temp;
}

int main()
{
    int a = 2, b = 3;
    fn( a, b );
    cout << a << " " << b;
    return 0;
}
```

A. 2 3

B. 2 2

 C. 3 2

D. 3 3

E. error

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString() {
        return "Point";
    };
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

void show(Point p){
    cout<<p.toString()<<endl;
}

int main()
{
    Circle c1;
    show(c1);
    return 0;
}
```

slido



What is the output of the previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString() {
        return "Point";
    };
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

- ✓ A. Point
- B. Circle
- C. error

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

void show(Point p){
    cout<<p.toString()<<endl;
}

int main()
{
    Circle c1;
    show(c1);
    return 0;
}
```

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString() {
        return "Point";
    };
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

void show(Point &p){
    cout<<p.toString()<<endl;
}

int main()
{
    Circle c1;
    show(c1);
    return 0;
}
```

slido



What is the output of the previous slide?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString() {
        return "Point";
    };
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

- A. Point
-  B. Circle
- C. error

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

void show(Point &p){
    cout<<p.toString()<<endl;
}

int main()
{
    Circle c1;
    show(c1);
    return 0;
}
```

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point() {_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString() {
        return "Point";
    };
    int getX() {
        return _x;
    }
    int getY() {
        return _y;
    }
};
```

```
class Circle:public Point{
    int _r;
public:
    Circle() {_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR() {
        return _r;
    }
};

void show(Point *p) {
    cout<<p->toString()<<endl;
}

int main()
{
    Circle c1;
    show(&c1);
    return 0;
}
```

What is the output of the following code?

```
#include <iostream>
using namespace std;

class Point{
    int _x, _y;
public:
    Point(){_x =1; _y =1;}
    Point(int x, int y):_x(x), _y(y){}
    virtual string toString() {
        return "Point";
    };
    int getX(){
        return _x;
    }
    int getY(){
        return _y;
    }
};
```

- A. Point
- ✓ B. Circle
- C. error

```
class Circle:public Point{
    int _r;
public:
    Circle(){_r =1;}
    Circle(int r):_r(r){}
    Circle(int r, int x, int y):Point(x,y),_r(r){}
    string toString(){
        return "Circle";
    }
    int getR(){
        return _r;
    }
};

void show(Point *p){
    cout<<p->toString()<<endl;
}

int main()
{
    Circle c1;
    show(&c1);
    return 0;
}
```

slido

Please download and install the
Slido app on all computers you use



Which of the following statements is true about
method overriding in C++ when using pointers
and references?

① Start presenting to display the poll results on this slide.

References are less powerful than pointers

- 1) Once a reference is created, it cannot be later made to reference another object; it cannot be reset. This is often done with pointers.
- 2) References cannot be NULL. Pointers are often made NULL to indicate that they are not pointing to any valid thing.
- 3) A reference must be initialized when declared. There is no such restriction with pointers.



TOPIC

Topic 8: Header Files

Interface/Implementation

- **Header files (.h)** is used for function **declarations**: declare the **interfaces**.
- **Source code (.cpp)** is used for defining class/function implementation.

```
// in myclass.h
class MyClass {
private :
    int value ;
public:
    void foo() ;
    int evaluate() ;
};
```

Interface Declaration
(header)

```
// in myclass.cpp #include
"myclass.h"

void MyClass::foo() {
    ....// implementation
}

int MyClass:: evaluate() {
    ....
}
```

Implementation
(implementation
source code)

```
// in main.cpp
#include "myclass.h"

int main() {
    MyClass a;
    .....// usage
    return 0;
}
```

Using the Class

slido

Please download and install the
Slido app on all computers you use



In C++, the separation of declaration (in header files) and implementation (in source files) provides several advantages. Which of the following is NOT one of those advantages?

① Start presenting to display the poll results on this slide.

1. Encapsulation

- Separating the interface (declaration) from the implementation (definition) enforces encapsulation. Users of a class or function only need to know about the interface, which is what the declaration provides. They don't need to know the details of how the class or function is implemented, which can remain hidden in the implementation file. This concept is key in object-oriented programming, as it allows changes to the implementation without affecting code that uses the interface.

2. Compile-Time Efficiency

- In a large project, it's common for only a small part of the code to change at a time. If the implementation is separate from the declaration, then only the implementation files (.cpp files) that have changed need to be recompiled, not the entire project. This can significantly reduce the compilation time, since the header files (.h or .hpp files) that only contain declarations generally don't need to be recompiled if they haven't changed.

Interface/Implementation: Java

- Java does not have separate header and source files like C++. Instead, Java uses .java files for both declaration and implementation. However, you can still separate the declaration and implementation to some extent using interfaces and abstract classes.
- You can include attributes in a Java interface, but they are implicitly public, static, and final, which essentially makes them constants.

Interface/Implementation: Advantages of Header Files

- It **speeds up compile time**. As your program grows, so does your code, and if everything is in a **single file**, then everything **must be fully recompiled** every time you make any little change.

```
#ifndef FILENAME_H  
#define FILENAME_H  
//...your header code here  
#endif
```

Include Guard

- It keeps your code more **organised**. If you **separate concepts** into specific files, it's **easier to find the code** you are looking for when you want to make modifications (or just look at it to remember how to use it and/or how it works).
- It allows you to separate *interface* from *implementation*. Client classes **only need** the interface files **(.h)** and **not** implementation files **(.cpp)**.

The preprocessor is a program that takes your program, makes some changes (for example include files (`#include`), macro expansion (`#define`), and basically everything that starts with `#`) and gives the "clean" result to the compiler.

The preprocessor works like this when it sees `#include`:

When you write:

```
#include "some_file"
```

The contents of `some_file` almost literally get copy pasted into the file including it.

Does the following program work fine?

```
a.h:  
class A { int a; };
```

```
b.h:  
#include "a.h"  
class B { int b; };
```

```
main.cpp:  
#include "a.h"  
#include "b.h"
```

slido



Does the program in previous slide work fine?

① Start presenting to display the poll results on this slide.

The following program works fine.

```
a.h:  
class A { int a; };
```

```
b.h:  
#include "a.h"  
class B { int b; };
```

```
main.cpp:  
#include "a.h"  
#include "b.h"
```

A. True

 B. False

```
a.h:  
class A { int a; };
```

```
b.h:  
#include "a.h"  
class B { int b;  
};
```

```
main.cpp:  
#include "a.h"  
#include "b.h"
```



```
main.cpp:  
class A { int a; }; // From #include "a.h"  
class A { int a; }; // From #include "b.h"  
class B { int b; }; // From #include "b.h"
```

Include Guard

a.h:

```
#ifndef A_H
#define A_H
class A { int a; };
#endif
```

b.h:

```
#ifndef B_H
#define B_H
#include "a.h"
class B { int b; };
#endif
```

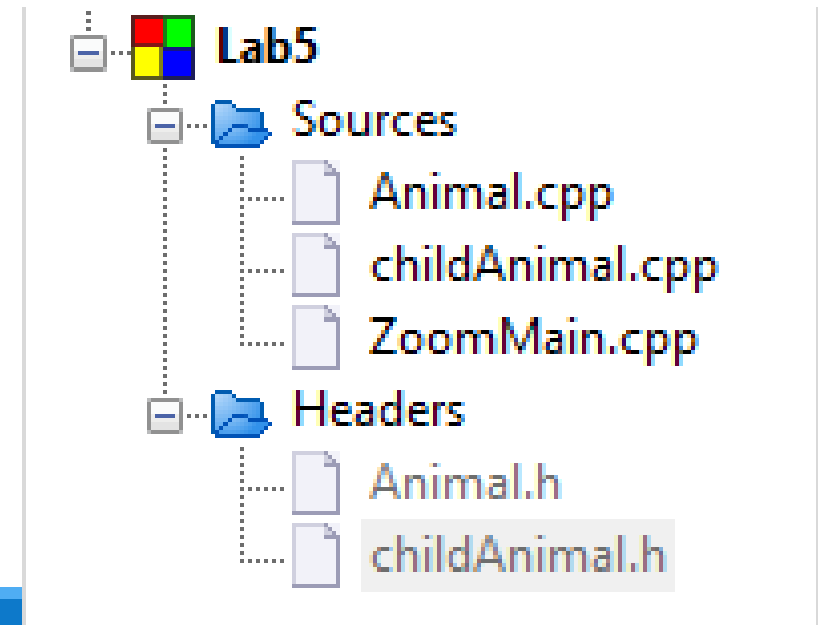
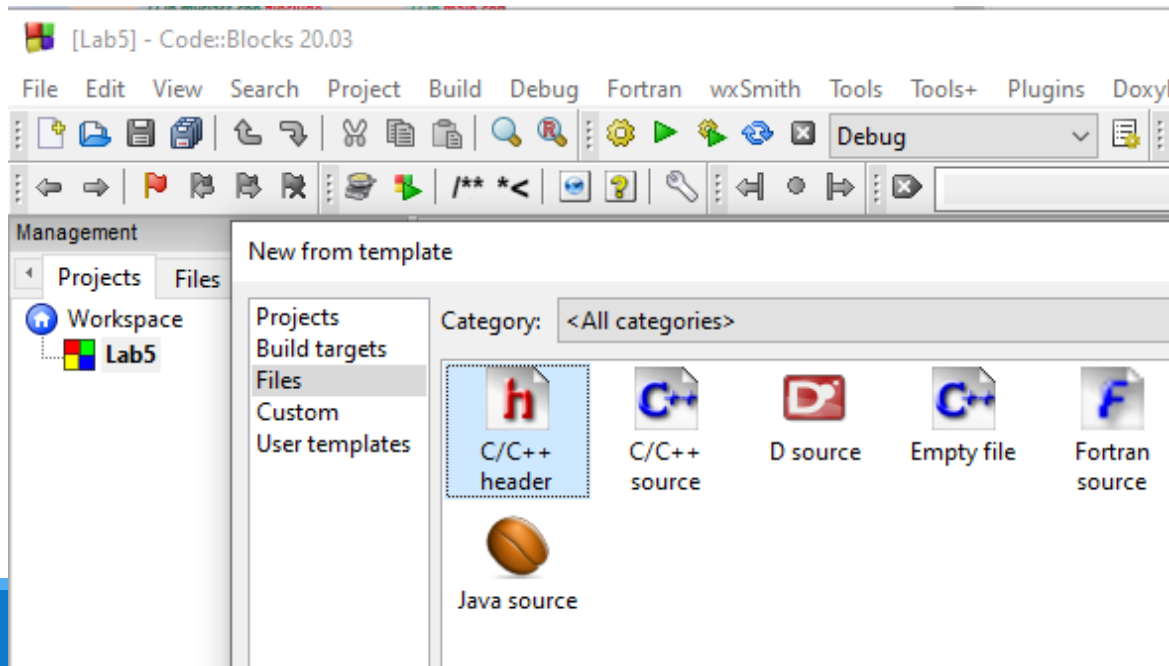
main.cpp:

```
// From #include "a.h"
#ifndef A_H
#define A_H
class A { int a; };
#endif
// From #include "b.h"
#ifndef B_H
#define B_H
// From
//
#include "a.h"
class A { int a; }; // inside
#endif // "b.h"
class B { int b; };
#endif
```

1. Header and Implementation files

- Separate the declaration code and Implementation code – create a header file/s (.h) for the declaration code and implementation file/s (.cpp) for the implementation code.

Eg, *animal.h* for Animal and Mammal class, *childAnimal.h* for Dog, Cat, etc, subclasses and a *ZooMain.cpp* for implementation. *Make sure you #include the header file/s accordingly.*



slido



What is the output of the following code?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

```
#include <iostream>
using namespace std;
class Student {
public:
    string name;
    int rollno;
    Student() {}
    Student(string x, int y) {
        name = x;
        rollno = y;
    }
    void printDetails() {
        cout << rollno << " - " << name << "; ";
    }
};

int main() {
    Student students[2];
    students[0] = Student("Ram", 5);
    students[1] = Student("Alex", 1);

    for(int i=0; i < 2; i++) {
        students[i].printDetails();
    }
}
```

Main.cpp

- ✓ A. 5 - Ram; 1 - Alex;
- B. 5 - 1 ;
- C. 5 – Ram 1 – Alex
- D. Error

What is the output of the following code?

Main.cpp

```
#include <iostream>
using namespace std;
int main() {
    Student students[2];
    students[0] = Student("Ram", 5);
    students[1] = Student("Alex", 1);

    for(int i=0; i < 2; i++) {
        students[i].printDetails();
    }
}
```

student.cpp

```
#include <iostream>
using namespace std;
class Student {
    public:
    string name;
    int rollno;
    Student() {}
    Student(string x, int y) {
        name = x;
        rollno = y;
    }
    void printDetails() {
        cout << rollno << " - " << name << "; ";
    }
};
```

slido



What is the output of the previous code?

① Start presenting to display the poll results on this slide.

What is the output of the following code?

Main.cpp

```
#include <iostream>
using namespace std;
int main() {
    Student students[2];
    students[0] = Student("Ram", 5);
    students[1] = Student("Alex", 1);

    for(int i=0; i < 2; i++) {
        students[i].printDetails();
    }
}
```

student.cpp

```
#include <iostream>
using namespace std;
class Student {
    int rollno;
    Student() {}
    Student(string x, int y) {
        name = x;
        rollno = y;
    }
    void printDetails() {
        cout << rollno << " - " << name << "; ";
    }
};
```

error: 'Student' was not declared in this scope |

A. 5 - Ram; 1 - Alex;

B. 5 - 1 ;

C. 5 – Ram 1 – Alex

✓ D. Error

Interface/Implementation

- **Header files (.h)** is used for function **declarations**: declare the **interfaces**.
- **Source code (.cpp)** is used for defining class/function implementation.

```
// in myclass.h
class MyClass {
    private :
        int value ;
    public:
        void foo() ;
        int evaluate() ;
};
```

Interface Declaration
(header)

```
// in myclass.cpp #include
"myclass.h"

void MyClass::foo() {
    ....// implementation
}

int MyClass:: evaluate() {
    ....
}
```

Implementation
(implementation
source code)

```
// in main.cpp
#include "myclass.h"

int main() {
    MyClass a;
    .....// usage
    return 0;
}
```

Using the Class

student.h

```
#ifndef STUDENT_H_INCLUDED
#define STUDENT_H_INCLUDED

#include <iostream>

using namespace std;

class Student {
public:
    string name;
    int rollno;

    Student();

    Student(string x, int y);

    void printDetails();
};
#endif // STUDENT_H_INCLUDED
```

```
#include <iostream>
#include "student.h"
using namespace std;

Student::Student() {}

Student::Student(string x, int y):name(x), rollno(y){}

void Student::printDetails() {
    cout << rollno << " - " << name << endl;
}
```

student.cpp

Main.cpp

```
#include <iostream>
#include "student.h"
using namespace std;

int main() {
    Student students[8];
    students[0] = Student("Ram", 5);
    students[1] = Student("Alex", 1);

    for(int i=0; i < 4; i++) {
        students[i].printDetails();
    }
}
```

Interface

- **Interfaces** are nothing but a way to describe the behavior of a class without committing to the implementation of the class. In C++ programming there is no built-in concept of interfaces. In order to create an interface, we need to create an abstract class which is having only pure virtual methods. In C++, Interfaces are also called pure abstract classes.

<https://www.geeksforgeeks.org/cpp-program-to-create-an-interface/>

Pure Virtual Functions

A Pure Virtual Function is a function where we only declare the function but not the function definition. The implementation for pure virtual methods is done at the derived class by method/function overriding. A function is said to be a pure virtual function if it is defined in the class as follows:

virtual datatype functionName(parameter1, parameter2,...) = 0

Step 1: Define the Interface in a Header File

```
// Drawable.h

#ifndef DRAWABLE_H
#define DRAWABLE_H

class Drawable {
public:
    virtual void draw() const = 0; // Pure
    virtual function making Drawable an
    abstract class
    virtual ~Drawable() {} // Virtual
    destructor is important for a base class
};

#endif // DRAWABLE_H
```

Step 2: Implement the Interface in Different Classes

```
// Circle.h

#include "Drawable.h"
#include <iostream>

class Circle : public Drawable {
public:
    void draw() const override {
        std::cout << "Drawing a circle." <<
std::endl;
    }
};
```

```
// Square.h

#include "Drawable.h"
#include <iostream>

class Square : public Drawable {
public:
    void draw() const override {
        std::cout << "Drawing a square." <<
std::endl;
    }
};
```

The override keyword is a part of modern C++ best practices, as it makes the code more robust and easier to understand and maintain. (NOT EXAMINABLE)

Step 3: Use the Interface

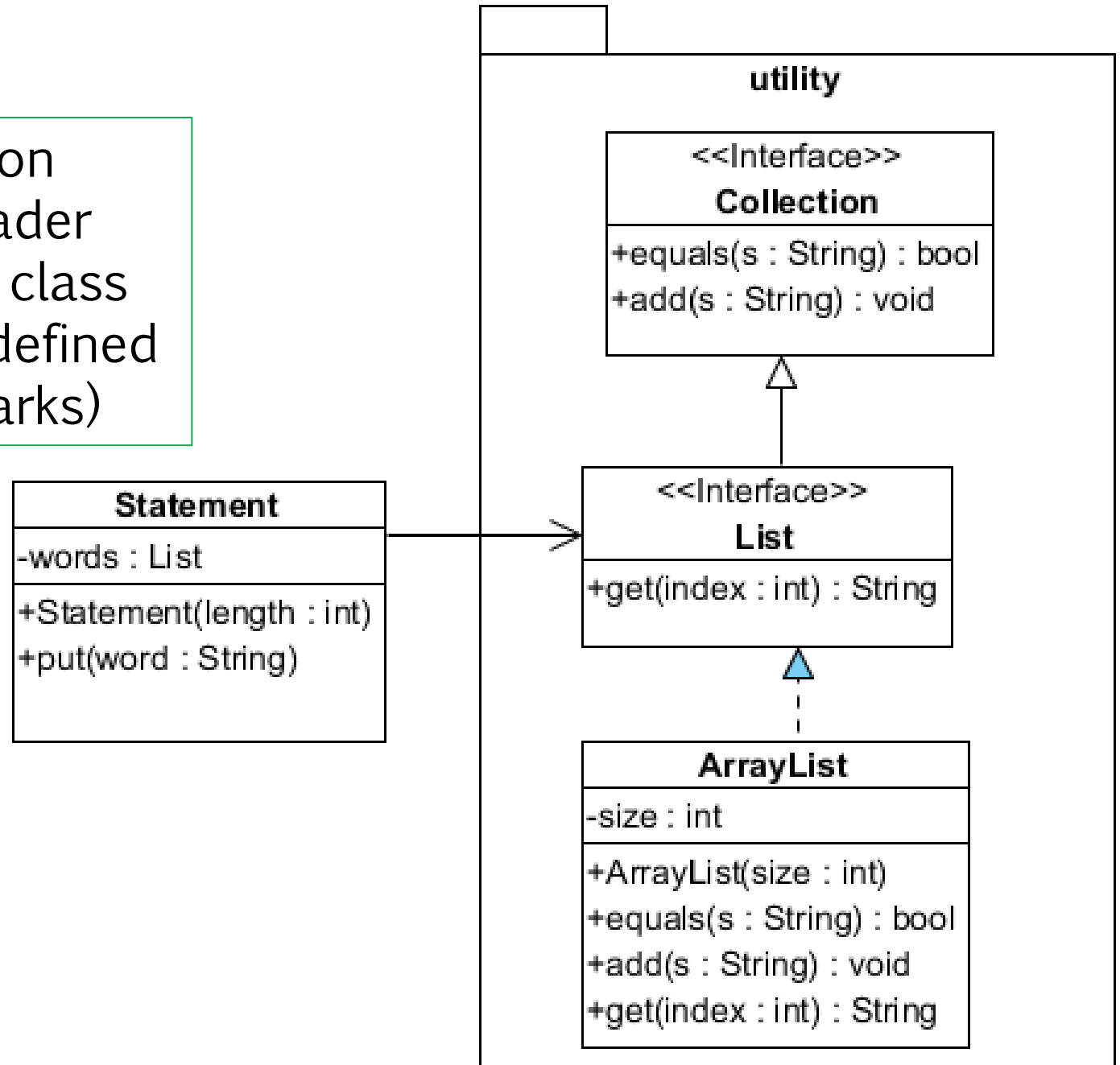
You can then use these classes polymorphically through pointers or references to the Drawable interface.



Past Year Paper
Revision

2017-2018S1

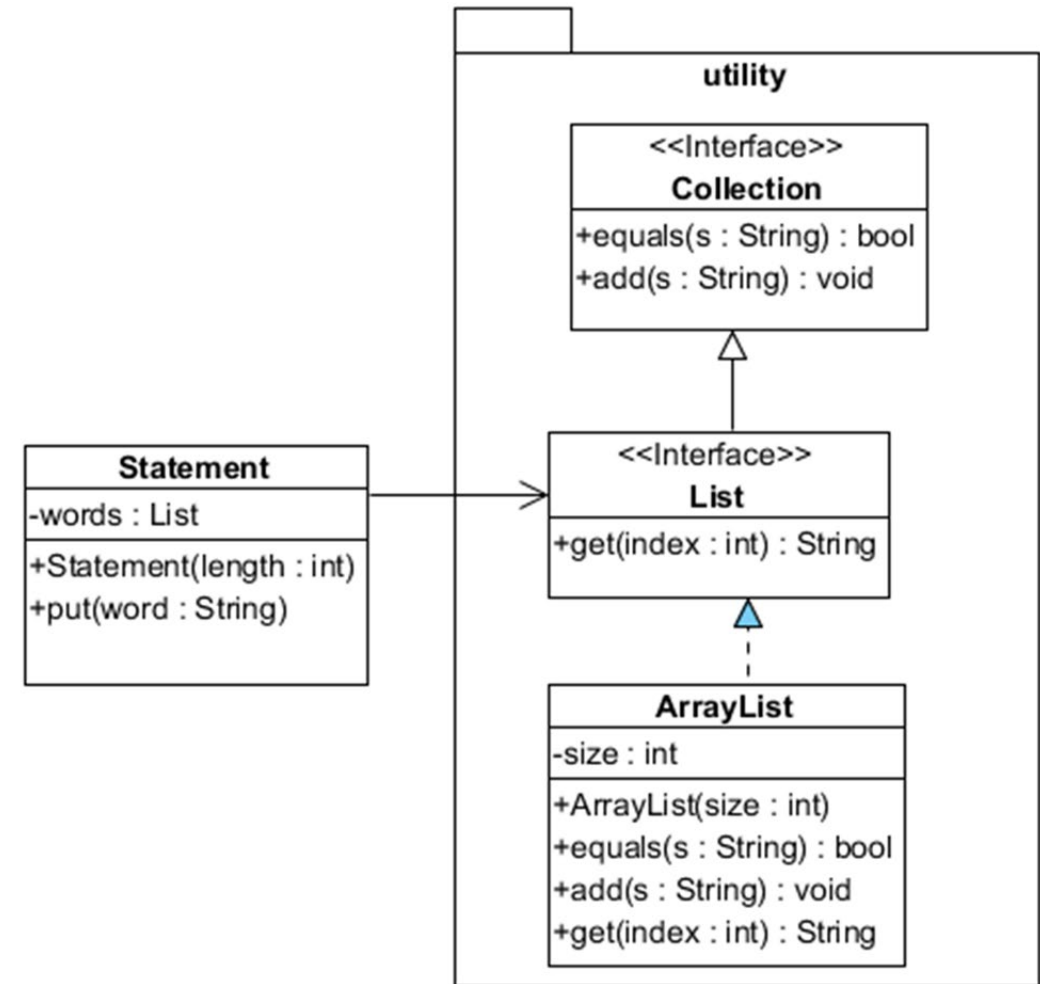
Write the C++ code for the Collection interface and List interface in a header file, collection.h, as depicted in the class diagram. The interfaces are to be defined in a C++ equivalent package. (7 marks)



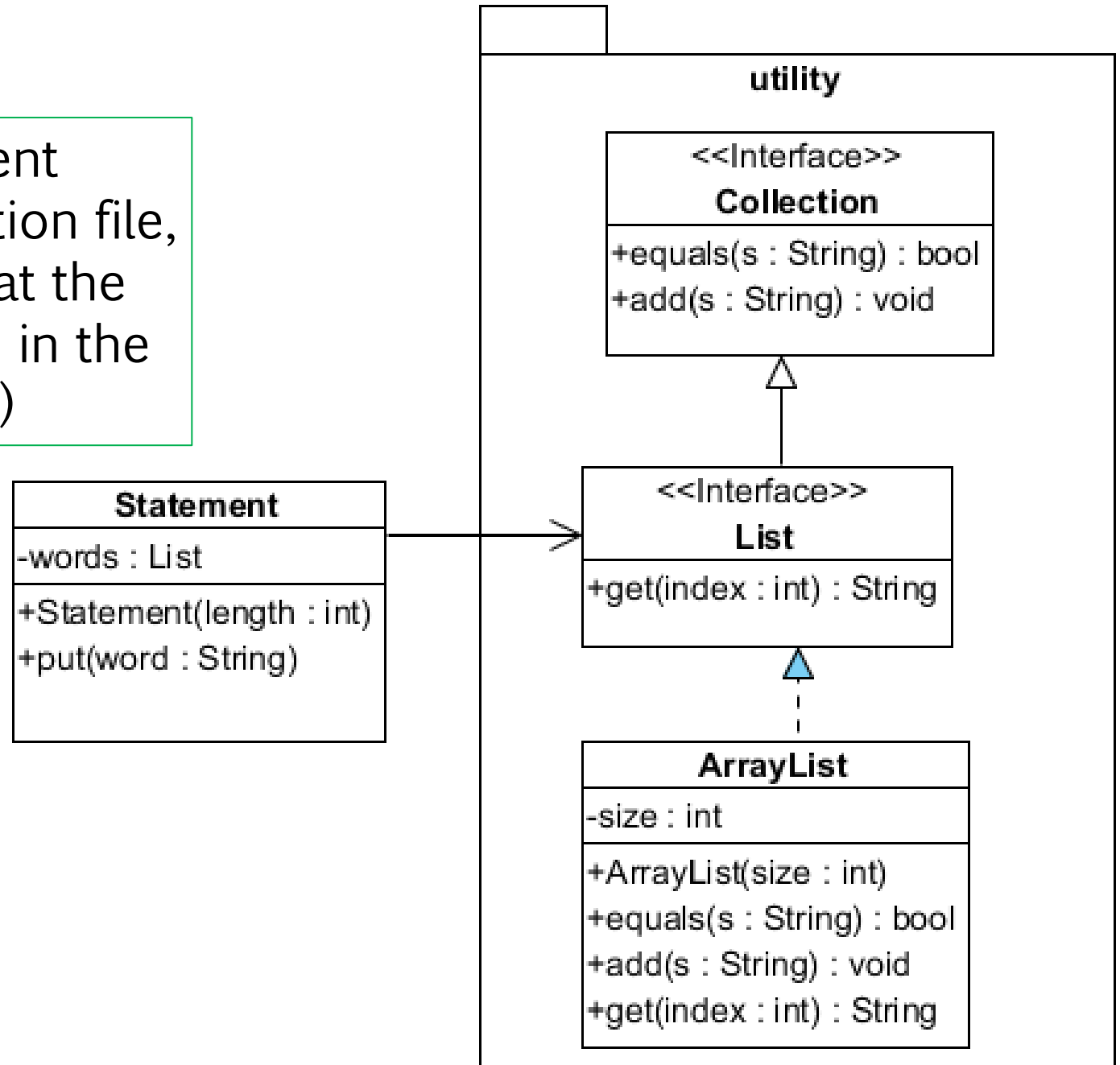
collection.h

```
#include <string> // 1
using namespace std ; // 1
namespace utility { // 2
    class Collection {
        public : // 1
            virtual bool equals(string s) =0; // 1
            virtual void add(string s) =0;
    };

    class List : public Collection { // 1
        public :
            virtual string get(int index) =0;
    };
}
```



Write the C++ code for the Statement class completely in an implementation file, statement.cpp. You can assume that the ArrayList class is also implemented in the collection.h header file. (6 marks)



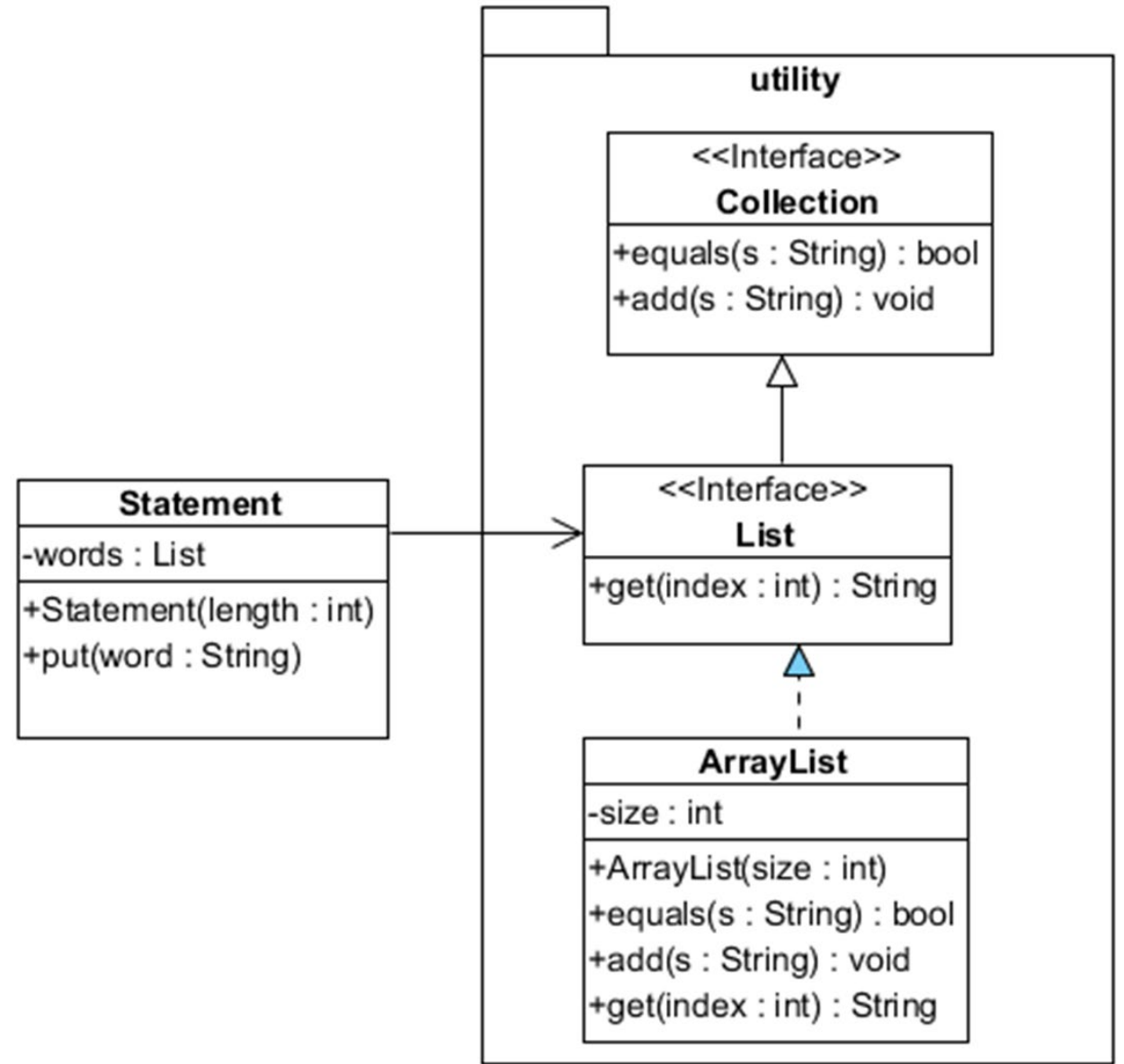
```

#include "Collection.h" // 1

using namespace utility; // 1

class Statement {
private :
    List *words; // 2
public :
    Statement(int s) {
        words = new ArrayList(s); // 1
    }
    void put(string word) {
        words ->add(word); // 1
    }
};

```





TOPIC

Topic 9: Advanced topic: Operator Overloading

slido



What is the output of the following program?

① Start presenting to display the poll results on this slide.

What is the output of the following program?

```
#include<iostream>
using namespace std;
class Complex {
private:
    int real, imag;
public:
    Complex(int r = 0, int i =0):real(r),imag(i) {}
};

int main()
{
    Complex c1(10, 5), c2(2, 4);
    c1 = c2;
    if (c1 == c2)
        cout << "Same";
    else
        cout<<"Different";
    return 0;
}
```

A. Same

B. Different

 C. Error

error: no match for 'operator==' (operand types are 'Complex' and 'Complex')

Assign operator is by default available in all user defined classes even if user has not implemented.

```
int main()
{
    Complex c1(10, 5), c2(2, 4), c3;
    c1 = c2;
    c3=c1 + c2;
    if (c1 == c2)
        cout << "Same";
    else
        cout<<"Different";
    return 0;
}
```

addition operator “+” is not overloaded.
comparison operator “==” is not overloaded.

Class **Operator** Overloading

Write a **Complex Number** class in **Java** such that you can perform addition of 2 complex numbers and return a result (a new complex number). Test it by adding 3 complex numbers.

```
public class ComplexNo {  
    private double real, imag ;  
    public ComplexNo(double r, double i) { real = r ; imag = i ;}  
    public ComplexNo add(ComplexNo c) {  
        return new ComplexNo(real + c.real, imag + c.imag);  
    }  
}
```

```
// usage :   ComplexNo c1 = new ComplexNo( 1.0, 2.0) ;  
            ComplexNo c2 = new ComplexNo( 2.0, 3.0) ;  
            ComplexNo c3 = new ComplexNo( 3.0, 4.0) ;  
            ComplexNo c4 = c1.add(c2).add(c3);
```

c4=c1+c2+c3

Class **Operator** Overloading

```
class { // in C++
    double _real, _imag;
public:
    Complex() : _real(0.0), _imag(0.0) {}
    Complex(const double real, const double imag) : _real(real), _imag(imag) {}
    Complex add(const Complex op) { /* the usual */ }
    Complex mul(const Complex op);
    ...
};
```



```
Complex c1(3,4), c2(4,5), c3(6,7), c4;
c4 = c1.add(c2).add(c3);
}
```

$c4 = c1 + c2 + c3$

Class **Operator** Overloading

```
class Complex {  
    double _real, _imag;  
    public:  
        Complex() : _real(0.0), _imag(0.0) {}  
        Complex(const double real, const double imag) : _real(real), _imag(imag) {}  
        Complex operator +(const Complex op) {  
            double real = _real + op._real, imag = _imag + op._imag;  
            return(Complex(real, imag));  
        }  
        Complex operator *(const Complex op) {...}  
        ...  
};
```

// evaluate ((c1 + c2) * (c1 + c2)) * c3
// where c1, c2, c3 are complex numbers
// demo.....



Operator overloading is a crucial concept in C++ that lets you achieve the functionality of the built-in operators while working with user-defined data types. Comparison operators in C++ are the ones that are there to compare two values with each other such as “==”, “!=”, “>”, “<”, “>=”, and “<=”.

```
Complex operator +(const Complex op) { ...}  
Complex operator *(const Complex op) {....}
```

```
class className { ... ..  
  
public:  
returnType operator symbol (arguments) { ... .. }  
... .. };
```

Class **Operator** Overloading

- *Overload* almost all of its operators for newly created types (classes).

<<		+=	!	!=	==
<	>	<=	>=	++	-- ...

- In previous case, **operator +** is a **member** of class *Complex*. An expression of the form:

`c = a + b;` where `a`, `b`, `c` are of *Complex* class is translated into a method call:

`c = a.operator+(b);`

`Complex a(3,4), b(4,5), c ;`

`c = a + b;`



```
// C++ program to overload the binary operator +
// This program adds two complex numbers

#include <iostream>
using namespace std;

class Complex {
private:
    float real;
    float imag;

public:
    // Constructor to initialize real and imag to 0
    Complex() : real(0), imag(0) {}

    void input() {
        cout << "Enter real and imaginary parts respectively: ";
        cin >> real;
        cin >> imag;
    }

    // Overload the + operator
    Complex operator + (const Complex& obj) {
        Complex temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }
}
```

```
void output() {
    if (imag < 0)
        cout << "Output Complex number: " << real <<
        imag << "i";
    else
        cout << "Output Complex number: " << real <<
        "+" << imag << "i";
    }
};
```

```
int main() {
    Complex complex1, complex2, result;

    cout << "Enter first complex number:\n";
    complex1.input();

    cout << "Enter second complex number:\n";
    complex2.input();

    // complex1 calls the operator function
    // complex2 is passed as an argument to the function
    result = complex1 + complex2;
    result.output();
    return 0;
}
```

Open



ComparisonOperator.cpp

~

```
#include <iostream>
using namespace std;
class ProductPrice
{
    private:
        int price;
    public:
        void userInput()
        {
            cin>>price;
        }
        bool operator ==(const ProductPrice &p)
        {
            if (price==p.price)
            {
                return 1;
            }
            else
            {
                return 0;
            }
        }
};
```

Open



ComparisonOperator.cpp

~

```
#include <iostream>
using namespace std;
class ProductPrice
{
    private:
        int price;
    public:
        void userInput()
        {
            cin>>price;
        }
        bool operator !=(const ProductPrice &p)
        {
            if (price!=p.price)
            {
                return 1;
            }
            else
            {
                return 0;
            }
        }
};
```

Class **Operator** Overloading

- **operator +** is **NOT** a member of class *Complex*

```
class Complex {  
    double _real, _imag;  
public:  
    Complex(const double real, const double imag):  
        _real(real), _imag(imag) {}  
    double real() { return _real; }  
    double imag() { return _imag; }  
};
```

// standalone function

```
Complex operator +(Complex op1, Complex op2) {  
    double real = op1.real() + op2.real();  
    double imag = op1.imag() + op2.imag();  
    return(Complex(real, imag));  
}
```

```
Complex operator +(const Complex op) {  
    double real = _real + op._real;  
    double imag = _imag + op._imag;  
    return(Complex(real, imag));  
}
```

Friends

Friend allows **non-member** function access to **private** data of a class.

```
class Complex {  
    double _real, _imag; // private  
public:  
    ....  
    friend Complex operator +( const Complex , const Complex );  
};  
Complex operator +(const Complex op1, const Complex op2) {  
    double real = op1._real + op2._real ;  
    double imag = op1._imag + op2._imag;  
    return(Complex(real, imag));  
}
```



friend

Non-member functions of a class will not have access to the private data of another class. There could be situations where we want two classes to share some functions and the data members. In that case, we can make the function a friend of these classes, and that will enable the function to access the private and protected data members of the classes.

```
class className{  
    // Other Declarations  
    friend returnType functionName(arg list);  
};
```

friend

A friend function can be:

- a) A member of another class
- b) A global function

```
class Node {  
private:  
    int key;  
    Node* next;  
  
    /* Other members of Node Class */  
    friend int LinkedList::search();  
    // Only search() of linkedList  
    // can access internal members  
};
```

Friend class

Friend Class A friend class can access private and protected members of other class in which it is declared as friend. It is sometimes useful to allow a particular class to access private members of other class.

```
// friend class SomeClass;
```

```
class Node {  
private:  
    int key;  
    Node* next;  
    /* Other members of Node Class */  
  
    // Now class LinkedList can  
    // access private members of Node  
    friend class LinkedList;  
};
```

- **Should not use friend unless necessary**
 - Break the data hiding principle.
 - If used often it is a sign that it is time to restructure your inheritance.

The background of the slide features a dense field of white, three-dimensional letters and numbers floating at various depths and angles, creating a sense of motion and complexity. Overlaid on this background is the word "TOPIC" in a large, bold, red 3D font. The letters of "TOPIC" are thick and have a slight shadow, making them stand out prominently from the chaotic letter background.

TOPIC

Topic 10: const

Class Inheritance

```
class Point3D : public Point {
```

```
    int _z;
```

```
public:
```

```
    Point3D( const int x, const int y, const int z) : Point(x, y)
```

```
        { _z = z; }
```

```
.....
```

```
};
```

Specify the desired base constructors after a **single colon** just before the body of constructor.

Using Base class constructor



```
: Point(x, y) , _z(z) { }
```

Class Multiple Inheritance

```
class DrawableString : public Point, public DrawableObject
```

```
{
```

```
.....
```

```
}
```

- **Const** can make **variables, pointers, references, and member functions** immutable.
 - using const in function parameters in C++ makes the parameter read-only within the function.
 - Const Objects
- It enhances code safety, readability, and helps avoid unintended changes to data.
- Proper use of const can lead to better optimization by the compiler.

Constant **variables**

Declares a variable whose value cannot be changed after initialization.

slido



What is the output of the following program?

① Start presenting to display the poll results on this slide.

What is the output of the following program?

```
int main() {  
    const int i = 5;  
    i = 10;  
    cout<<i<<endl;  
}
```

A. 5

B. 10

✓ C. Error

The const keyword specifies that a variable's value is constant and tells the compiler to prevent the programmer from modifying it.

const vs #define

```
#define WIDTH      80
```

```
const int WIDTH    80
```

In C++, you can use the `const` keyword instead of the `#define` preprocessor directive to define constant values. Values defined with `const` are subject to type checking, and can be used in place of constant expressions.

Constant Pointers

Pointer to a Const Value: The value pointed to by the pointer cannot be changed, but the pointer itself can be reassigned.



What does the following
declaration mean?
`const int* ptr;`

① Start presenting to display the poll results on this slide.

Const Pointer: The pointer itself cannot be reassigned, but the value it points to can be changed.

slido

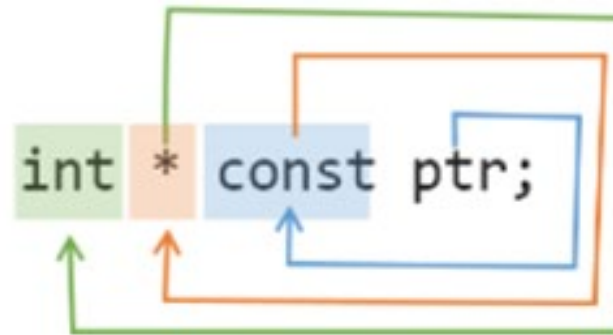


What is the output of the following program?

① Start presenting to display the poll results on this slide.

What is the output of the following program?

```
int main() {  
    char this_char{'a'}, that_char{'b'};  
    char *mybuf = &this_char, *yourbuf = &that_char;  
    char *const aptr = mybuf;  
    *aptr = 'c';  
    cout<<*mybuf<<endl;  
}
```



ptr is a `const` pointer to `int`

- A. a
- B. b
- ✓ C. c
- D. Error

slido

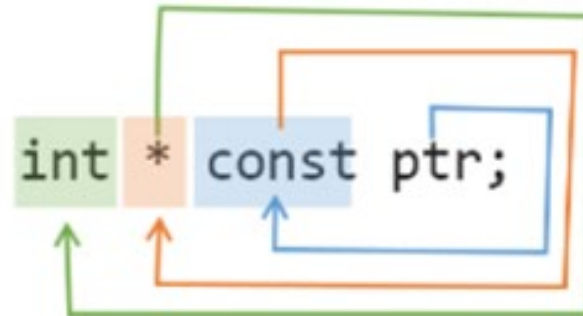


What is the output of the following program?

① Start presenting to display the poll results on this slide.

What is the output of the following program?

```
int main() {  
    char this_char{'a'}, that_char{'b'};  
    char *mybuf = &this_char, *yourbuf = &that_char;  
    char *const aptr = mybuf;  
    *aptr = 'c';  
    cout<<*mybuf<<endl;  
    aptr = yourbuf;  
}
```



ptr is a **const** **pointer** to **int**

A. a

B. b

C. c

✓ D. Error

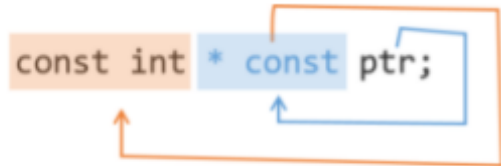
For those who don't know about Clockwise/Spiral Rule: Start from the name of the variable, move clockwise (in this case, move backward) to the next **pointer** or **type**. Repeat until expression ends.

Here is a demo:



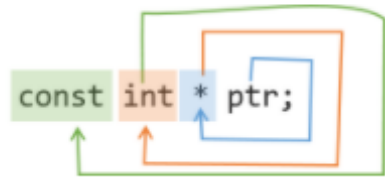
```
int * ptr;
```

ptr is a **pointer** to **int**



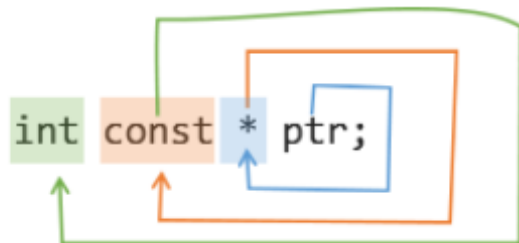
```
const int * const ptr;
```

ptr is a **constant pointer** to **const int**



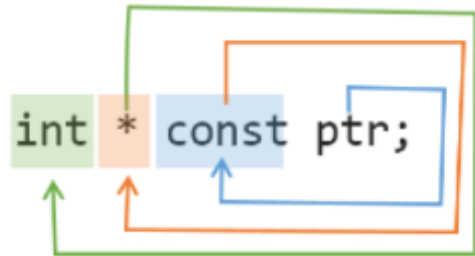
```
const int * ptr;
```

ptr is a **pointer** to **int constant** (i.e. const int)



```
int const * ptr;
```

ptr is a **pointer** to **const int**



```
int * const ptr;
```

ptr is a **const pointer** to **int**

If the word const appears to the left of the asterisk, what's pointed to is constant
If the word const appears to the right of the asterisk, the pointer itself is constant
If const appears on both sides, both are constant

From C to C++

■ **const**

- to declare particular aspects of a variable (**or object**) to be constant
- **const** variable
- Examples,

int i; // just an ordinary integer

int *ip; // uninitialised pointer to integer

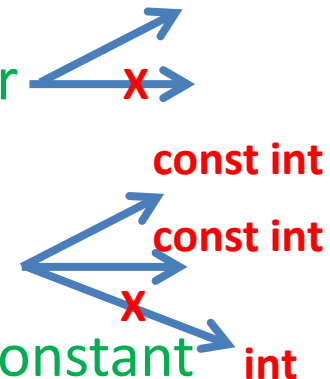
int * const cp = &i; // **constant pointer** to integer

const int ci = 7; // constant integer

const int *cip; // **pointer to constant integer**

const int * const cicp = &ci; // **constant pointer to constant** int

const int // integer



Const in Function Parameters

using const in function parameters in C++ makes the parameter read-only within the function.

What is the output of the following program?

```
void foo(const int y)
{
    cout << y;
}

void fool(int y)
{
    y = 5;
    cout << y;
}
```

```
int main()
{
    int x = 9;
    const int z = 10;

    foo(z);
    fool(z);
    cout << z;
    return 0;
}
```

slido



What is the output of the previous program?

① Start presenting to display the poll results on this slide.

What is the output of the following program?

```
void foo(const int y)
{
    cout << y;
}

void fool(int y)
{
    y = 5;
    cout << y;
}
```

```
int main()
{
    int x = 9;
    const int z = 10;

    foo(z);
    fool(z);
    cout << z;
    return 0;
}
```

- A. 101010
-  B. 10510
- C. 1055
- D. Error

What is the output of the following program?

```
void foo(const int y)
{
    y = 6;
    cout << y;
}

void fool(int y)
{
    y = 5;
    cout << y;
}
```

```
int main()
{
    int x = 9;
    const int z = 10;

    foo(z);
    fool(z);
    cout << z;
    return 0;
}
```

slido



What is the output of the previous program?

① Start presenting to display the poll results on this slide.

What is the output of the following program?

```
void foo(const int y)
{
    y = 6;
    cout << y;
}

void fool(int y)
{
    y = 5;
    cout << y;
}
```

If it is passed in as a constant, whatever is passed in as a value can't be changed within the function. You can only read it but cannot modify it.

A. 101010

B. 10510

C. 1055

✓ D. Error

```
int main()
{
    int x = 9;
    const int z = 10;

    foo(z);
    fool(z);
    cout << z;
    return 0;
}
```

Const Reference Parameters

When you pass a parameter as a const reference, it prevents the function from modifying the argument. This is useful for passing large objects to avoid copying while ensuring they aren't changed.

Example

```
void display(const std::string& text) {  
    // text is read-only here  
  
    std::cout << text << std::endl;  
  
    // text = "new value"; // Error: cannot modify a const  
reference  
}
```

- using & makes our code efficient by referencing the complex2 object instead of making a duplicate object inside the operator function.
- using const is considered a good practice because it prevents the operator function from modifying complex2.

```
class Complex {  
    ... ..  
    public:  
    ... ..  
    Complex operator +(const Complex& obj) {  
        // code  
    }  
    ... ..  
};  
  
int main() {  
    ... ..  
    result = complex1 + complex2;  
    ... ..  
}
```

The diagram illustrates a function call from the `main` function to the `operator +` function. A teal line starts from the `complex1` variable in the `main` function, goes down and then right, ending with an arrow pointing to the `Complex` parameter of the `operator +` function. Another teal arrow points from the `complex2` variable in the `main` function to the `obj` parameter of the `operator +` function.

function call from complex1

Const Pointer Parameters

When a function parameter is a pointer to const, the data being pointed to cannot be modified via the pointer.

Example

```
void printValue(const int* ptr) {  
    std::cout << *ptr << std::endl;  
    // *ptr = 5; // Error: cannot modify the value via  
    const pointer  
}
```

Summary

- const references prevent the function from modifying the argument, while avoiding the cost of copying (especially useful for large objects).
- const pointers ensure the function cannot change the data being pointed to.
- const value parameters ensure the value cannot be changed within the function, but it's a copy, not the original argument.

Constant member function

Declares that a member function will not modify the object's state (i.e., it won't change member variables).

What is the output of the following program?

```
class Test {  
    int value;  
  
public:  
    Test(int v = 0) { value = v; }  
  
    int getValue(int x ) const {x =9; cout<<x;  return value; }  
};  
  
int main()  
{  
    Test t(20);  
    cout << t.getValue(7);  
    return 0;  
}
```

slido



What is the output of the previous program?

① Start presenting to display the poll results on this slide.

What is the output of the following program?

```
class Test {  
    int value;  
  
public:  
    Test(int v = 0) { value = v; }  
  
    int getValue(int x ) const {x =9; cout<<x;  return value; }  
};  
  
int main()  
{  
    Test t(20);  
    cout << t.getValue(7);  
    return 0;  
}
```

- A. 720
- ✓ B. 920
- C. 2020
- D. Error

What is the output of the following program?

```
class Test {  
    int value;  
  
public:  
    Test(int v = 0) { value = v; }  
  
    int getValue(int x ) const {x =9; cout<<x; value = x; return value; }  
};  
  
int main()  
{  
    Test t(20);  
    cout << t.getValue(7);  
    return 0;  
}
```

slido



What is the output of the previous program?

① Start presenting to display the poll results on this slide.

What is the output of the following program?

```
class Test {
    int value;

public:
    Test(int v = 0) { value = v; }

    int getValue(int x ) const {x =9; cout<<x; value = x; return value; }
};

int main()
{
    Test t(20);
    cout << t.getValue(7);
    return 0;
}
```

A. 720

B. 920

C. 99

✓ D. Error

If it is a part of the method but have constant behind, it is usually for a case where it is declared within a class itself. Within this method that is declared as constant method or function you cannot make modifications to its attribute. You can read the attribute values, but you cannot change the values inside.

const member functions

- Declaring a member function with the const keyword specifies that the function is a "read-only" function that doesn't modify the object for which it's called.
- A constant member function can't modify any non-static data members or call any member functions that aren't constant.
- To declare a constant member function, place the const keyword after the closing parenthesis of the argument list. The const keyword is required in both the declaration and the definition.

Const Objects

When an object is declared as const, only const member functions can be called, ensuring the object's state cannot be changed.

Example

```
class MyClass {  
    int value;  
public:  
    int getValue() const { return value; } // Const  
member function  
};
```

```
const MyClass obj;  
  
obj.getValue(); // Allowed  
  
obj.setValue(10); // Error if setValue is not  
a const function
```

The background of the slide features a dense field of white, three-dimensional letters and numbers floating at various depths and angles, creating a sense of motion and complexity. Overlaid on this background is the word "TOPIC" in a large, bold, red 3D font. The letters of "TOPIC" are thick and have a slight shadow, making them stand out prominently from the chaotic letter background.

TOPIC

Topic 11: Destructor

Class Destructors

```
class Point {  
    int _x, _y;    // point coordinates, default private  
public:  
    Point() : _x(0), _y(0) {}  
    Point(const int x, const int y) : _x(x), _y(y) {}  
  
    ~Point() { /* do clean up */ } // Destructor  
    //prefixed by a tilde (~) of the defining class  
    .....  
};
```

Ensure that the allocated memory is released!

Destructor is an instance member function which is invoked automatically whenever an object is going to be destroyed. Meaning, a destructor is the last function that is going to be called before an object is destroyed.

How are destructors different from a normal member function?

- Destructors have same name as the class preceded by a tilde (~)
- Destructors don't take any argument and don't return anything

When do we need to write a user-defined destructor?

- If we do not write our own destructor in class, compiler creates a default destructor for us.
- The default destructor works fine unless we have dynamically allocated memory or pointer in class.
- The object is created by using new or the constructor uses new to allocate memory which resides in the heap memory or the free store, the destructor should use delete to free the memory. This must be done to avoid memory leak.

```
#include<iostream>
#include <string.h>
using namespace std;
```

```
class String {
private:
```

```
    char* s;
    int size;
```

```
public:
```

```
    String(char*); // constructor
    ~String(); // destructor
```

```
};
```

```
String::String(char* c)
```

```
{
```

```
    size = strlen(c);
```

```
    s = new char[size + 1];
```

```
    strcpy(s, c);
```

```
    cout<<s<<c;
```

```
}
```

```
String::~~String() { delete[] s; }
```

```
int main() {
```

```
    String s("abcdef");
```

```
}
```

delete keyword in C++

- Delete is an ***operator*** that is used to **destroy** **array** and **non-array(pointer) objects** which are created by new expression.
- Delete can be used by either using ***Delete operator*** or ***Delete [] operator***
 - New operator is used for dynamic memory allocation which puts variables on heap memory.
 - Which means Delete operator deallocates memory from heap.

delete

```
#include <iostream>

int main() {
    int* ptr = new int; // Allocation for a single int
    *ptr = 42; // Assigning value to allocated int
    std::cout << "Value: " << *ptr << std::endl;

    delete ptr; // Correctly using delete for single object
    ptr = nullptr; // Good practice to avoid dangling pointer

    return 0;
}
```

Delete[]

```
#include <iostream>
int main() {
    int* arr = new int[5]; // Allocation for an array of 5 ints
    // Initializing array
    for(int i = 0; i < 5; ++i) {
        arr[i] = i * 10;
    }
    // Accessing array
    for(int i = 0; i < 5; ++i) {
        std::cout << "arr[" << i << "]: " << arr[i] << std::endl;
    }
    delete[] arr; // Correctly using delete[] for an array
    arr = nullptr; // Good practice to avoid dangling pointer
    return 0;
}
```

Setting the pointer to nullptr after deletion

```
int* ptr = new int(10); // Dynamically allocate an int
delete ptr; // Delete the allocated int
ptr = nullptr; // Prevent ptr from becoming a dangling pointer
```

By setting the pointer to nullptr after deletion, you clearly indicate that the pointer is no longer pointing to valid memory, helping to prevent errors in code that might accidentally dereference such pointers later on.

Always delete what you new

Always delete[] what you new[]

Key Points

- Always match new with delete and new[] with delete[].
- After deallocating memory, setting the pointer to nullptr is a good practice to avoid using dangling pointers.
- Failure to match the allocation and deallocation method (e.g., using delete on an array allocated with new[], or vice versa) results in undefined behavior.

DELETE

It is used to release the memory occupied by an object which is no longer needed

It releases memory held by a single object which is allocated using new operator

It calls a class's destructor once

Syntax:
`delete objName;`

DELETE[]

It is used to get rid of an array's pointer and release the memory occupied by the array.

It frees memory held by an array of an object which allocated using new[]

It calls as many destructors as the size of the array

Syntax:
`delete[] arrName;`



TOPIC

Topic 12: String

```
#include <stdio.h>
#include <string.h>
```

```
#define M1 "12345"
char M2[40] = "12345";
char M3[3] = "12345";
char *M4 = "12345";
char M5[] = "12345";
```

String in c

String Class in Java

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/String.html>

Module java.base

Package java.lang

Class String

java.lang.Object
 java.lang.String

All Implemented Interfaces:

Serializable, CharSequence, Comparable<String>

```
public final class String
extends Object
implements Serializable, Comparable<String>, CharSequence
```

The `String` class represents character strings. All string literals in Java programs are instances of the `String` class.

Strings are constant; their values cannot be changed after they are created. For example, the following code

```
String str = "abc";
```

is equivalent to:

```
char data[] = {'a', 'b', 'c'};
String str = new String(data);
```

Here are some more examples of how strings can be used:

```
System.out.println("abc");
String cde = "cde";
System.out.println("abc" + cde);
String c = "abc".substring(2,3);
```

int	<code>codePointAt(int index)</code>
int	<code>codePointBefore(int index)</code>
int	<code>codePointCount(int beginIndex, int endIndex)</code>
IntStream	<code>codePoints()</code>
int	<code>compareTo(String anotherString)</code>
int	<code>compareToIgnoreCase(String str)</code>
String	<code>concat(String str)</code>
boolean	<code>contains(CharSequence s)</code>
boolean	<code>contentEquals(CharSequence cs)</code>
boolean	<code>contentEquals(StringBuffer sb)</code>
static String	<code>copyValueOf(char[] data)</code>
static String	<code>copyValueOf(char[] data, int offset, int count)</code>
boolean	<code>endsWith(String suffix)</code>
boolean	<code>equals(Object anObject)</code>
boolean	<code>equalsIgnoreCase(String anotherString)</code>
static String	<code>format(String format, Object... args)</code>
static String	<code>format(Locale l, String format, Object... args)</code>

String Class in C++

#include <string>

Function	Task
append()	Appends a part of string to another string.
at()	Obtains the character stored at a specified location.
compare()	Compares string against the invoking string.
empty()	Returns true if the string is empty; otherwise returns false.
erase()	Removes character as specified.
find()	Searches for the occurrence of a specified substring.
insert()	Inserts character at specified location.
length()	Gives number of elements in a string.
replace()	Replace specified characters with a given string.

<http://www.cplusplus.com/reference/string/string/>

String Class

Creating `string` objects

```
string s1; // using constructor with no argument

string s2("xyz"); // using one-argument constructor

s1=s2 ;

s3 = "abc" + s2 ;

cin>> s1 ; // reading through keyboard a word

getline(cin,s1) ; // reading through keyboard a line of text.
```

String Class

Manipulating `string` objects

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s1("12345");
    string s2("abcde");
    for (int i=0; i < s1.length(); i++)
        cout << s1.at(i);
    for (int j=0; j < s2.length(); j++)
        cout << s2[j];

    s1.insert(4,s2); // insert a string s2 into s1 at position 4 : s1 = 1234abcde5
    s1.erase(4,5); // remove 5 characters from s1 starting position 4 : s1 = 12345
    s2.replace(1,3,s1); // replace 3 characters in s2 with s1 starting position 4:
                       // s2 = a12345e
}
```

```
#include <iostream>
#include <string>

int main() {
    std::string greeting = "Hello";
    std::cout << greeting;
    return 0;
}
```



TOPIC

Topic 13: Arrays

C++ Array Declaration

```
dataType arrayName[arraySize];
```

For example,

```
int x[6];
```

Here,

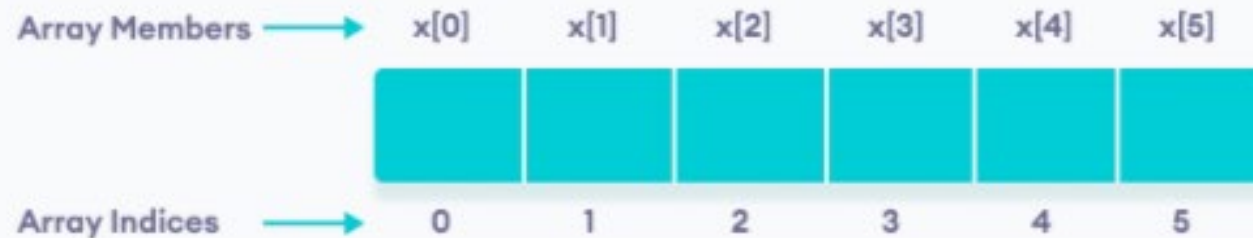
- `int` - type of element to be stored
- `x` - name of the array
- `6` - size of the array

Access Elements in C++ Array

In C++, each element in an array is associated with a number. The number is known as an array index. We can access elements of an array by using those indices.

```
// syntax to access array elements  
array[index];
```

Consider the array `x` we have seen above.



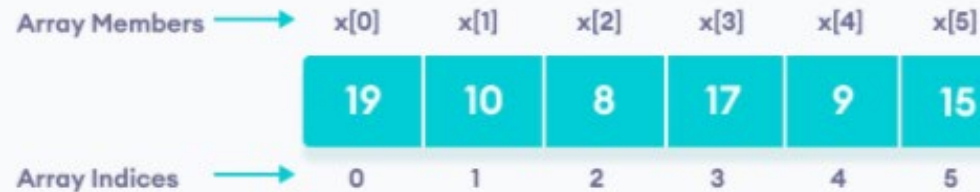
Elements of an array in C++

```
float figure[3] ;  
figure[0] = 0.79 ;
```

C++ Array Initialization

In C++, it's possible to initialize an array during declaration. For example,

```
// declare and initialize an array
int x[6] = {19, 10, 8, 17, 9, 15};
```



C++ Array elements and their data

Another method to initialize array during declaration:

```
// declare and initialize an array
int x[] = {19, 10, 8, 17, 9, 15};
```

Here, we have not mentioned the size of the array. In such cases, the compiler automatically computes the size.

slido



What is the output of the following program?

```
#include <iostream>
using namespace std;
int main() {
    double  figure[3] = {1.2,3.4,5,6};
    cout<<figure[2];
}
```

① Start presenting to display the poll results on this slide.

```
#include <iostream>
using namespace std;
int main() {
double  figure[3] = {1.2,3.4,5,6};
cout<<figure[2];
}
```

error: too many initializers for 'double [3]'

slido

What is the output of the following program?



```
#include <iostream>
using namespace std;
int main() {
    double  figure[4] = {1,6};
    for (int i=0; i<4;i++)
        cout<<figure[i];
}
```

① Start presenting to display the poll results on this slide.

C++ Array With Empty Members

In C++, if an array has a size `n`, we can store upto `n` number of elements in the array. However, what will happen if we store less than `n` number of elements.

For example,

```
// store only 3 elements in the array  
int x[6] = {19, 10, 8};
```

// less is ok, more is not

Here, the array `x` has a size of `6`. However, we have initialized it with only 3 elements.

In such cases, the compiler assigns random values to the remaining places. Oftentimes, this random value is simply `0`.



```
float figure[3] ;  
figure[0] = 0.79 ; // first element is always index 0  
figure[1] = 0.88;  
figure[2] = 0.32 ; // last element, since size is 3  
  
int a[4] = { 1,2,3,4} ;  
int b[] = { 1,2,3,4,5} ;  
int c[5] = {1,2 } ; // less is ok, more is not
```

Array of Objects

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int rollno;

    Student() {}

    Student(string x, int y) {
        name = x;
        rollno = y;
    }

    void printDetails() {
        cout << rollno << " - " << name << endl;
    }
};
```

C++ Array Declaration

```
dataType arrayName[arraySize];
```

```
int main() {
    //declare array with specific size
    Student students[5];

    //assign objects
    students[0] = Student("Ram", 5);
    students[1] = Student("Alex", 1);
    students[2] = Student("Leshu", 4);
    students[3] = Student("Emily", 3);
    students[4] = Student("Anita", 2);

    for(int i=0; i < 5; i++) {
        students[i].printDetails();
    }
}
```

```

#include <iostream>
using namespace std;

class Person {
public:
    virtual void disp() {
        cout<<"disp person";
    }
};

class Student:public Person {
    string _name;
    int _age;

public:
    Student(string name,int age):_name(name),_age(age) {}
    void disp()
    {
        cout << _name << "\t" << _age << "\n";
    }
};

int main()
{
    
    Student s("jack", 22);
    persons[0]= &s;
    persons[1]= new Student("Rose",20);
    persons[0]->disp();
    persons[1]->disp();
    return 0;
}

```

Fill in the blank to complete the program to generate the output as shown.

jack	22
Rose	20

slido



Fill in the blank to complete the program to generate the output as shown.

① Start presenting to display the poll results on this slide.

```
#include <iostream>
using namespace std;
```

```
class Person {
public:
    virtual void disp() {
        cout<<"disp person";
    }
};
```

```
class Student:public Person {
    string _name;
    int _age;

public:
    Student(string name,int age):_name(name),_age(age) {}
    void disp()
    {
        cout << _name << "\t" << _age << "\n";
    }
};
```

```
int main()
{
    
    Student s("jack", 22);
    persons[0]= &s;
    persons[1]= new Student("Rose",20);
    persons[0]->disp();
    persons[1]->disp();
    return 0;
}
```

C++ Array Declaration

```
dataType arrayName[arraySize];
```

jack	22
Rose	20

- Dynamic array (vector in C++, ArrayList in Java)

vector<datatype>array name;

vector<datatype>array name={values};

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> num {1, 2, 3, 4, 5};

    cout << "Element at Index 0: " << num.at(0) << endl;
    cout << "Element at Index 2: " << num.at(2) << endl;
    cout << "Element at Index 4: " << num.at(4);

    return 0;
}
```

Capacity:

<u>size</u>	Return size (public member function)
<u>max_size</u>	Return maximum size (public member function)
<u>resize</u>	Change size (public member function)
<u>capacity</u>	Return size of allocated storage capacity (public member function)
<u>empty</u>	Test whether vector is empty (public member function)
<u>reserve</u>	Request a change in capacity (public member function)
<u>shrink_to_fit</u>	Shrink to fit (public member function)

Element access:

<u>operator[]</u>	Access element (public member function)
<u>at</u>	Access element (public member function)
<u>front</u>	Access first element (public member function)
<u>back</u>	Access last element (public member function)
<u>data</u>	Access data (public member function)

Modifiers:

<u>assign</u>	Assign vector content (public member function)
<u>push_back</u>	Add element at the end (public member function)
<u>pop_back</u>	Delete last element (public member function)
<u>insert</u>	Insert elements (public member function)
<u>erase</u>	Erase elements (public member function)
<u>swap</u>	Swap content (public member function)

<https://cplusplus.com/reference/vector/vector/>

```
class Person {
    string _name;
    int _age;

public:
    Person(string name, int age): _name(name), _age(age) {}
    virtual void disp() {
        cout<<"disp person";
    }
};
```

```
class Student:public Person {
    string _matric;
public:
    Student(string name, int age, string matric): Person(name, age), _matric(matric) {}
    void disp()
    {
        cout <<"student " << _matric << "\n";
    }
};
```

```
class Staff:public Person {
    string _office;
public:
    Staff(string name, int age, string office): Person(name, age), _office(office) {}
    void disp()
    {
        cout <<"staff " << _office << "\n";
    }
};
```

Fill in the blank to complete the program to generate the output as shown.

```
int main()  
{  
      
    cout << "Initial size: " << persons.size() << endl;  
    persons.push_back(new Student("jack", 22, "U2234578H"));  
    Student s("Rose", 20, "U2166666H");  
    persons.push_back(&s);  
    persons.push_back(new Staff("Tom", 65, "ARC-04-01"));  
    cout << "New size: " << persons.size() << endl;  
    for (int i=0; i<persons.size(); i++)  
        persons[i]->disp();  
    return 0;  
}
```

Initial size: 0
New size: 3
student U2234578H
student U2166666H
staff ARC-04-01

slido



Fill in the blank to complete the program to generate the output as shown.

① Start presenting to display the poll results on this slide.

vector<datatype>array name;

Fill in the blank to complete the program to generate the output as shown.

```
int main()  
{  
      
    cout << "Initial size: " << persons.size() << endl;  
    persons.push_back(new Student("jack", 22, "U2234578H"));  
    Student s("Rose", 20, "U2166666H");  
    persons.push_back(&s);  
    persons.push_back(new Staff("Tom", 65, "ARC-04-01"));  
    cout << "New size: " << persons.size() << endl;  
    for (int i=0; i<persons.size(); i++)  
        persons[i]->disp();  
    return 0;  
}
```

Initial size: 0
New size: 3
student U2234578H
student U2166666H
staff ARC-04-01

When to Use Array or Vector

Use an Array When:

- The number of elements is known at compile time and won't change.
- You're working in constrained environments where the overhead of a vector is unacceptable.
- You need the utmost performance and are willing to manage memory manually for dynamic arrays.

Use a Vector When:

- The number of elements is unknown at compile time or might change over time.
- You want the convenience of automatic memory management and easy resizing.
- You prefer safety features like bounds checking.

Vectors are generally recommended for most situations due to their flexibility and safety features. Arrays might be more suitable for performance-critical or low-level programming tasks where the overhead of a dynamic size and automatic memory management is undesirable.



TOPIC

Topic 14: The Keyword “static”

- The static data members are basically same in Java and C++. The static data members are the property of the class, and it is shared to all of the objects.
- In C++ and Java, the static members can be accessed without creating some objects
- **Static variables in a Function (C++):** When a variable is declared as static, space for **it gets allocated for the lifetime of the program**. Even if the function is called multiple times, space for the static variable is allocated only once and the value of the variable in the previous call gets carried through the next function call.
 - Java doesn't allow static local variables in functions.

slido



what is the output?

① Start presenting to display the poll results on this slide.

```
#include <iostream>
using namespace std;

class A {
public:
    static int i;

    A() {
        i++;
    };
};

int A::i = 0;

int main()
{
    A obj;
    obj.i ++;
    A a;
    cout << a.i;
}
```

3

slido



What is the output?

① Start presenting to display the poll results on this slide.

```
#include <iostream>
using namespace std;

class A{
public:
    static void printMsg() {
        cout << "Print!"; }
};

int main()
{
    A a;
    a.printMsg();
    A::printMsg();
}
```

Print!Print!

Static variables in a Function

- A static variable is not deleted when the function in which it's defined finishes executing.
- In C++, a static variable inside a function retains its value between function calls. When you declare a variable as static within a function, it is initialized only once (the first time the function is called) and remains in memory until the program ends. Its value is preserved across multiple calls to that function.

slido



What is the output?

① Start presenting to display the poll results on this slide.

```
#include <iostream>
#include <string>
using namespace std;

void demo()
{
    static int count;
    count=8;
    cout << count << " ";
    count++;
}
```

```
int main()
{
    demo();
    cout << count << " ";
    return 0;
}
```

error: 'count' was not declared in this scope

slido



What is the output?

① Start presenting to display the poll results on this slide.

```
#include <iostream>
#include <string>
using namespace std;
```

```
void demo()
```

```
{
```

```
    static int count;
    cout << count << " ";
    count=8;
    cout << count << " ";
    count++;
```

```
}
```

```
int main()
```

```
{
```

```
    for (int i = 0; i < 2; i++)
        demo();
    return 0;
```

```
}
```

0 8 9 8



Topic 14: Java vs. C++



Everything in Java must be in a class. There are no global functions or global data.



There's no scope resolution operator `::` in Java. Java uses the dot for everything (including package). package (Java) vs. namespace (C++).



In Java, all objects of **non-primitive** types can be created only via **new**. There's no equivalent to creating non-primitive objects “on the stack” as in C++.



In Java, Object handle (references) defined as class members are **automatically initialised to null**. Initialization of primitive class data members is guaranteed in Java.



There are no Java pointers in the sense of C and C++. When you create an object with **new**, you get back a reference.



There are no destructors in Java.



Java uses a **singly-rooted hierarchy** inherited from the root class **Object**. In C++ you can start a new inheritance tree anywhere, so you end up with a forest of trees.

Java Comparison to C++ (OO aspects)

- § **Inheritance in Java has the same effect as in C++, but the syntax is different.** (however, the super keyword in Java allows you to access methods only in the parent class, one level up in the hierarchy. Base-class scoping (::) in C++ allows you to access methods that are deeper in the hierarchy).
- § **Java provides the interface keyword** which creates the equivalent of an abstract base class filled with abstract methods and with no data members.
- § **There's no virtual keyword in Java** because all non-static methods always use dynamic binding. In Java, the programmer doesn't have to decide whether or not to use dynamic binding. The reason virtual exists in C++ is so you can leave it off for a slight increase in efficiency when you're tuning for performance (or, put another way, "if you don't use it you don't pay for it").
- § **Java doesn't provide multiple inheritance (MI).**
- § **There is method overloading, but no operator overloading in Java.**

Java Comparison to C++ (OO aspects)

- C is primarily a subset of C++.
- Header files help in speeding up compile time, keep code more organised, and allow us to separate interface from implementation.
- The difference in Java and C++ with regards to OO are:
 - Everything in Java must be in a class.
 - There is no scope resolution operator in Java.
 - In Java, all objects of non-primitive types can be created only via new.
 - In Java, Object handle (references) defined as class members are automatically initialised to null.
 - There are no Java pointers in the sense of C and C++.
 - There are no destructors in Java.
 - Java uses a singly-rooted hierarchy.

Key Points